Highlights

Certified Compilation of Graph Coloring to Neutral-Atom Hardware: An Exact Gadgetized Reduction with a Heuristic Layout Optimizer

Mahmoud Naderi, Manuel Mazzara, Paolo Ciancarini

- Exact, typed reduction from graph coloring to unit-disk MWIS on Rydberg arrays
- Gap-ε solves of the combinatorial gadget cost at most ε extra colors
- Non-colorability verdicts certified by a checker proved sound in Rocq
- Gadget size linear in n for bounded-pathwidth graphs, quadratic in general
- Calculus, gadget, checker and end-to-end robustness mechanized in Rocq

Certified Compilation of Graph Coloring to Neutral-Atom Hardware: An Exact Gadgetized Reduction with a Heuristic Layout Optimizer

Mahmoud Naderi[a,*], Manuel Mazzara[b] and Paolo Ciancarini[c]

[a] *System and Software Engineering Lab, Innopolis University, Innopolis, Russia*
[b] *Faculty of Computer Science and Engineering, Innopolis University, Innopolis, Russia*
[c] *Department of Computer Science and Engineering, University of Bologna, Bologna, Italy*

ARTICLE INFO

Keywords:
graph coloring
unit-disk graph
maximum independent set
neutral-atom computing
certified compilation
type-and-effect system

ABSTRACT

In their analog Rydberg-blockade mode with the standard two-level encoding, neutral-atom processors natively solve maximum-weight independent set (MWIS) on unit-disk graphs (UDGs), not graph coloring. A coloring compiler for this regime must therefore reduce coloring to UDG–MWIS. We reduce in two exact stages: k-colorability to independent set on a color-choice graph, then this graph to a weighted UDG by the copy/crossing gadgets of Nguyen et al. (2023). Each stage is typed as a decoder-carrying reduction in a *certified-reduction type system*, so end-to-end soundness and the legal stage order follow from typing. We add three results. First, decoders carry additive slack grades that compose; hence, for k-colorable G, a solve of the combinatorial (non-unit-disk) gadget instance that misses the optimum by ε decodes to a proper coloring with at most $k+\varepsilon$ colors. Second, the device only searches for colorings; every non-colorability verdict of our separator decomposition comes with a refutation accepted by a checker proved sound in Rocq and extracted to OCaml. Third, for graphs of pathwidth p the gadget instance needs $O(nk^2(p+1))$ atoms, linear in n, against $O((nk)^2)$ in general. The calculus, the combinatorial gadget, the checker and the end-to-end robustness theorem are mechanized axiom-free in Rocq. A classical Rydberg simulator agrees with a brute-force oracle on 304 runs, and all 14 835 shots from imperfect sweeps decode within the bound. All 26 decomposition verdicts, including $\chi(\texttt{myciel3}) = 4$, are certified. Atom counts for paths and cycles grow linearly, by about 55 and 91 atoms per vertex, yet only the smallest instances fit a 256-atom device.

1. Introduction

Graph coloring is a canonical NP-complete problem Karp (1972); Garey and Johnson (1979); its applications include register allocation, scheduling and frequency assignment. Neutral-atom processors solve a related problem in hardware. Atoms excited to *Rydberg states* interact through the *Rydberg blockade* Saffman, Walker and Mølmer (2010); Henriet, Beguin, Signoles, Lahaye, Browaeys, Reymond and Jurczak (2020); Scholl, Schuler, Williams, Eberharter, Barredo, Schymik, Lienhard, Henry, Lang, Lahaye, Läuchli and Browaeys (2021), so the excited atoms form an independent set of the *unit-disk graph* (UDG) defined by the atom positions. In the ideal-blockade model, an array driven adiabatically in its *analog* mode settles into a *maximum-weight independent set* (MWIS) of that UDG, with weights set by per-atom *local detunings* Pichler, Wang, Zhou, Choi and Lukin (2018); Ebadi, Keesling, Cain, Wang, Levine, Bluvstein, Semeghini, Omran, Liu, Samajdar, Luo, Nash, Gao, Barak, Farhi, Sachdev, Gemelke, Zhou, Choi, Pichler, Wang, Greiner, Vuletić and Lukin (2022); de Oliveira, Diamond-Hitchcock, Walker, Wells-Pestell, Pelegrí, Picken, Malcolm, Daley, Bass and Pritchard (2025). Section 2 defines these terms and states this correspondence as an interface, which is the only point at which the physics enters the paper. The same devices also run a gate-based *digital* mode Graham, Song, Scott, Poole, Phuttitarn, Jooya, Eichler, Jiang, Marra, Grinkemeyer, Kwon, Ebert, Cherek, Lichtman, Gillette, Gilbert, Bowman, Ballance, Campbell, Dahl,

*Corresponding author
m.naderi@innopolis.university (M. Naderi); m.mazzara@innopolis.ru (M. Mazzara); paolo.ciancarini@unibo.it (P. Ciancarini)

Crawford, Blunt, Rogers, Noel and Saffman (2022). We restrict the paper to the analog mode for three reasons: it realizes the blockade-to-MWIS correspondence natively, without gate-level compilation; its errors come from a single adiabatic sweep rather than accumulating gate by gate; and the cited hardware is publicly accessible in this mode Wurtz, Bylinskii, Braverman, Amato-Grill, Cantu, Huber, Lukin, Liu, Weinberg, Long, Wang, Gemelke and Keesling (2023). The compilation problem is therefore the following:

> *In the analog regime, the hardware solves MWIS on a unit-disk graph. It does not color. Any pipeline that compiles coloring to this hardware must reduce coloring to UDG–MWIS; embedding the input graph geometrically is not enough.*

Two designs. *Design A*, a geometric supergraph embedding, places the vertices of G in the plane and chooses r so that the induced UDG H is a *supergraph* of G on the same vertices, $E(G) \subseteq E(H)$. Any proper coloring of H then restricts to a proper coloring of G, so the *decoder* is a single restriction. However, we prove (Theorem 5.5) that such an H cannot bound *chromatic-number inflation*, the excess of $\chi(H)$ over $\chi(G)$: the star $K_{1,n}$ has $\chi = 2$, yet every UDG supergraph of it has $\chi(H) \geq \lceil n/5 \rceil + 1$. Moreover, even a supergraph with $\chi(H) = \chi(G)$ cannot be executed, because the device does not color graphs. *Design B*, an exact gadgetized reduction, reduces k-colorability of G to a single MWIS instance by the classical colorability→independent-set reduction and embeds that instance into a weighted UDG with the copy/crossing *gadgets* of Nguyen, Liu, Wurtz, Lukin, Wang and Pichler (2023). Its correctness is a meta-theorem proved once for all inputs: the chromatic number is not inflated, and the decoder is exact. For a given input, only two questions remain open: whether the instance fits the device, and whether the analog dynamics reaches the ground state.

We adopt Design B. Design A is retained in two roles, which we keep apart throughout. First, it serves as a *stand-alone supergraph backend*, kept only for comparison and never used for a certified answer. Second, its seven geometric heuristics act, inside Stage 2 of Design B, as a *layout optimizer* for the fixed adjacency of the gadget instance; in this role a poor layout can make the instance infeasible but cannot produce a wrong answer.

Contributions.

(1) An exact two-stage reduction from k-colorability to UDG–MWIS with an end-to-end exactness and decoding meta-theorem (Theorem 4.6) and $O((nk)^2)$ size bounds; an impossibility result that rules out the supergraph route as a correctness mechanism (Theorem 5.5); and seven polynomial-time layout heuristics with termination guarantees (Section 5).

(2) A coloring-specification language and a *certified-reduction type system* (Section 8): composing decoder-carrying stages gives end-to-end soundness (Theorem 8.3), the legal stage order follows from typing (Corollary 8.4), and refinements track certificate grade, supergraph safety, decomposition, and graded decoders.

(3) *Robustness to imperfect solves*: decoders carry additive slack grades that compose. As a result, if G is k-colorable, a solve that misses the optimum of the combinatorial gadget instance by ε decodes to a proper coloring with at most $k + \varepsilon$ colors (Theorem 4.10, mechanized as a single Rocq theorem). This additive guarantee complements the known result that such reductions do not preserve approximation ratios.

(4) *Certificates in both directions*: each verdict of a separator decomposition is emitted either as a coloring or as a refutation tree, and a checker proved sound in Rocq and extracted to OCaml accepts it. Colorings thus come from the device, and lower bounds come from classical refutations checked by a verified program (Section 9.8).

(5) A *pathwidth size bound*: we lift the $O(N \cdot \mathrm{pw})$ lattice bound of Nguyen et al. (2023, App. A) to G'_k and show that the gadget instance of a graph of pathwidth p needs $O(nk^2(p+1))$ atoms, linear in n, under stated assumptions on how the lines are placed (Proposition 4.12).

(6) An implementation and evaluation (Section 9): an axiom-free Rocq mechanization of the calculus, the combinatorial gadget, the checkers and Theorem 4.10; a prototype Haskell front end; and validation against a brute-force oracle and a classical Rydberg simulator. The validation covers imperfect sweeps, metric layouts, and a certified decomposition that establishes $\chi(\texttt{myciel3}) = 4$.

Artifact and structure. The Rocq development, the Haskell package `udg-color`, the simulator, and all benchmark scripts and results are publicly available at `https://github.com/slant14/Graph-colouring`. Sections 2–3 give the background and the pipeline; Sections 4–6 cover the exact path and the heuristic layout; Section 7 reviews related work; Section 8 presents the type system; Section 9 reports the mechanization and evaluation; and Sections 10–12 discuss limitations and conclude.

2. Background

This section defines the physical and compilation vocabulary in computer-science terms. A reader who accepts the interface stated below can read the rest of the paper as combinatorics and geometry.

Atoms, the blockade, and unit-disk graphs. An *atom* is one neutral atom held at a chosen position in the plane by an optical tweezer; the compiler emits atoms, and each is a vertex of the graph the device realizes. Atoms in bijection with the vertices of the abstract instance are *logical*; the extra atoms a construction adds are *ancillas*. Each atom is used as a two-level system with a ground state $|g\rangle$ and a *Rydberg state* $|r\rangle$, a highly excited electronic state Saffman et al. (2010); we read "excited" as the Boolean 1. Two excited atoms interact through a van der Waals shift that decays as the inverse sixth power of their distance, so strongly that, to a good approximation, two atoms within the *blockade radius* r cannot both be excited, while atoms further apart are effectively independent Saffman et al. (2010); Henriet et al. (2020); Scholl et al. (2021). Like the works we build on Pichler et al. (2018); Nguyen et al. (2023), we adopt this sharp threshold as the *ideal-blockade model*. A *placement* $p : V \to \mathbb{R}^2$ and a radius r define the *unit-disk graph* $\mathrm{UDG}(p, r)$ (Definition 3.1); in this model, the set of simultaneously excited atoms is always an independent set of $\mathrm{UDG}(p, r)$.

Ground states, detunings, and imperfect sweeps. The array is driven by a *Rabi drive* Ω, which transfers atoms between $|g\rangle$ and $|r\rangle$, and a *detuning* Δ, which sets how strongly the excited state is favored. A *local detuning* Δ_v, set per atom, acts as a vertex weight Pichler et al. (2018). Site-resolved light shifts implementing such weights have been demonstrated on a Rydberg array de Oliveira et al. (2025). QuEra's Aquila is specified with a global detuning Wurtz et al. (2023); since 2024 it has also offered, as an experimental capability, a static per-atom pattern $h_v \in [0, 1]$ that scales one shared, time-dependent local detuning Komar, Bylinskii, Becker and Lin (2024). The device reaches its lowest-energy configuration, the *many-body ground state*, *adiabatically*: the controls are swept slowly, starting from a state that is easy to prepare. A sweep that is too fast leaves the array in a *low-lying excited state*, which corresponds combinatorially to an independent set that is not of maximum weight. This is the only physical failure mode the paper must handle (Remark 4.7, Section 4.4); the rest of the physics enters through the following interface:

> **Interface.** In the ideal-blockade model, with positive local detunings w and the drive switched off at the end, the many-body ground state of atoms placed at p with blockade radius r is a maximum-weight independent set of $\mathrm{UDG}(p, r)$ under w; with a uniform detuning it is a maximum independent set Ebadi et al. (2022).

Compilation vocabulary. A *palette* is the set of k available colors, identified with $\{1, \ldots, k\}$. Because hardware couples only nearby atoms, an abstract graph cannot be realized directly: a *gadget* is a small, fixed pattern of atoms and weights that implements one constraint with short-range interactions only, and *gadgetization* replaces the vertices and edges of an abstract instance by gadgets (Section 4.2); the result is the weighted unit-disk *gadget instance*. A *decoder* is a polynomial-time map from solutions of a transformed instance back to solutions of the original one.

Reductions that carry decoders. A many-one reduction equipped with a polynomial-time witness translation is a *Levin reduction* Levin (1973); when the translation is a bijection between solution sets, so that their number is preserved, the reduction is *parsimonious* Simon (1975); Valiant (1979). The reduction of Theorem 4.2 is a Levin reduction. Its bijection between the size-n independent sets of the color-choice graph and the proper k-colorings also makes it parsimonious on those sets, so it reduces counting k-colorings

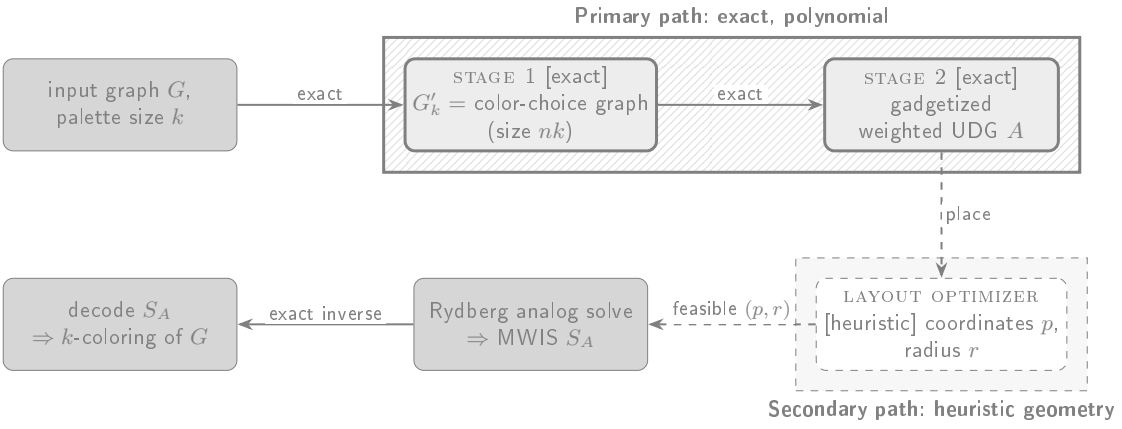


Figure 1: The two-layer pipeline. Correctness depends only on the primary path (thick borders, hatched panel); the secondary path (dashed borders) affects only hardware feasibility and the quality of the analog solve.

to counting independent sets of size n. The embedding of Theorem 4.4 also carries a decoder. Neither notion is a contribution of this paper; Section 8 contributes the treatment of such a reduction as a *type*.

3. Problem setup and pipeline overview

We consider finite simple undirected graphs $G = (V, E)$ with $n = |V|$ and $m = |E|$.

Definition 3.1 (Coloring; unit-disk graph; MWIS). (i) A *proper k-coloring* of G is a map $c : V \to \{1, \dots, k\}$ with $c(u) \neq c(v)$ for every edge $\{u, v\} \in E$; the *chromatic number* $\chi(G)$ is the least k for which one exists. (ii) Given a placement $p : V \to \mathbb{R}^2$ and a radius $r > 0$, the *unit-disk graph* $\mathrm{UDG}(p, r)$ has vertex set V and an edge $\{u, v\}$, $u \neq v$, iff $\|p(u) - p(v)\|_2 \leq r$; a weighted UDG also carries weights $w : V \to \mathbb{R}_{>0}$. (iii) An *independent set* is a set $S \subseteq V$ spanning no edge; $\mathrm{MWIS}(G, w) = \max\{\sum_{v \in S} w(v) : S \text{ independent}\}$, and for unit weights this is the independence number $\alpha(G)$.

Notation. $G[S]$ is the subgraph induced by $S \subseteq V$; $N(v)$ and $N[v] = N(v) \cup \{v\}$ are the open and closed neighborhoods; $\Delta(G)$ is the maximum degree, $\omega(G)$ the clique number, and $\sigma(G) = \max_v \alpha(G[N(v)])$ the size of the largest independent set inside a neighborhood. The device constraints are a minimum atom spacing $d_{\min}$, a maximum blockade radius $r_{\max}$, a finite array area $A_{\max}$, and an atom budget $N_{\max}$ Wurtz et al. (2023).

The two-layer pipeline. Figure 1 shows the design. The primary, exact path reduces (G, k) to the color-choice graph G'_k (Stage 1) and gadgetizes it into a weighted UDG instance A (Stage 2). The secondary, heuristic path chooses atom coordinates and a radius for A that obey the hardware constraints. The device returns an independent set of A, and the decoder maps it back through both stages to a coloring of G. The stand-alone supergraph backend never lowers χ (Proposition 5.2) but may raise it without bound (Theorem 5.5).

4. Primary path: exact gadgetized reduction

The primary path is the composition of two exact reductions:

$$\underbrace{G, k}_{\text{coloring}} \xrightarrow{\text{Stage 1}} \underbrace{G'_k}_{\text{MIS instance, size } nk} \xrightarrow{\text{Stage 2}} \underbrace{A = (p, r, w)}_{\text{weighted UDG}} \xrightarrow{\text{solve}} S_A \xrightarrow{\text{decode}} \text{coloring of } G.$$

4.1. Stage 1: colorability → independent set

Definition 4.1 (Color-choice graph). Given $G = (V, E)$ and palette size k, the *color-choice graph* is the graph $G'_k = (V', E')$ with unit weights, vertex set $V' = V \times \{1, \dots, k\}$, and edge set $E' = E'_{\text{choice}} \cup E'_{\text{conflict}}$

where

$$E'_{\text{choice}} = \big\{\{(v,i),(v,j)\} : v \in V,\ 1 \le i < j \le k\big\}, \qquad \text{(a } k\text{-clique per vertex)}$$
$$E'_{\text{conflict}} = \big\{\{(u,i),(v,i)\} : \{u,v\} \in E,\ 1 \le i \le k\big\}. \qquad \text{(same-color conflicts)}$$

Then $|V'| = nk$ and $|E'| = n\binom{k}{2} + mk$.

G'_k is the Cartesian product $G \,\square\, K_k$ Hammack, Imrich and Klavžar (2011): the copies of K_k are the choice cliques, and the copies of G carry the conflict edges.

Read $(v,i) \in V'$ as the *slot* "vertex v takes color i". The choice cliques allow at most one slot per vertex, and the conflict edges forbid equal colors at the two ends of an edge of G; nothing else is forbidden. An independent set of G'_k is therefore exactly a partial proper coloring of G (Figure 3 (b)), total iff it has size n.

Theorem 4.2 (Exactness of Stage 1). *For every graph G and every $k \ge 1$,*

$$\alpha(G'_k) = n \quad \Longleftrightarrow \quad G \text{ is } k\text{-colorable},$$

and $\alpha(G'_k) \le n$ always. The size-n independent sets of G'_k are in bijection with the proper k-colorings of G.

Proof. For each v, the k vertices $\{(v,i)\}_i$ form a clique (E'_{choice}), so any independent set S contains at most one slot per original vertex; hence $|S| \le n$.

($\Leftarrow$) Let c be a proper k-coloring. Put $S = \{(v, c(v)) : v \in V\}$, so $|S| = n$. S is independent: it has exactly one slot per vertex (no choice edge is internal), and for any $\{u,v\} \in E$ we have $c(u) \ne c(v)$, so no conflict edge $\{(u,c(u)),(v,c(v))\}$ exists, because conflict edges join only *equal* color indices. Thus $\alpha(G'_k) \ge n$, and combined with $\alpha(G'_k) \le n$ we get $\alpha(G'_k) = n$.

($\Rightarrow$) Let S be independent with $|S| = n$. By the choice-clique bound S has exactly one slot per vertex, defining $c(v) = i$ where $(v,i) \in S$. For $\{u,v\} \in E$, if $c(u) = c(v) = i$ then $\{(u,i),(v,i)\} \in E'_{\text{conflict}}$ would be internal to S, contradicting independence. Hence c is proper.

The two maps $c \mapsto S$ and $S \mapsto c$ are mutually inverse, giving the bijection.

The least k with $\alpha(G'_k) = n$ is $\chi(G)$; since $\chi(G) \le n$, binary search finds it with $O(\log n)$ solves.

Restrictions. Restrictions of the language (Section 8) on single vertices or pairs of vertices are independence constraints on G'_k, so Theorem 4.2 extends to them.

Proposition 4.3 (Restrictions). *Let every vertex v have a set $L(v) \subseteq \{1,\dots,k\}$ of allowed colors, and let D and Q be sets of pairs of distinct vertices that must receive different and equal colors, respectively. Delete from G'_k every slot (v,i) with $i \notin L(v)$; add the edges $\{(u,i),(w,i)\}$ for every $\{u,w\} \in D$; and add the edges $\{(u,i),(w,j)\}$ with $i \ne j$ for every $\{u,w\} \in Q$. The size-n independent sets of the resulting graph are in bijection with the proper k-colorings c of G such that $c(v) \in L(v)$ for every v, $c(u) \ne c(w)$ for every pair in D, and $c(u) = c(w)$ for every pair in Q.*

Proof. As in the proof of Theorem 4.2, a size-n independent set holds one slot per vertex and defines a proper coloring c. The deleted slots are exactly the disallowed colors; an edge added for $\{u,w\} \in D$ joins two slots exactly when they give u and w the same color; and an edge added for $\{u,w\} \in Q$ joins two slots exactly when they give u and w different colors. Conversely, a coloring that satisfies the restrictions selects no deleted slot and no two ends of an added edge.

In the language, `precolor`, `allow` and `forbid` define L, `differentColor` and `distinctOn` define D, and `sameColor` defines Q. The restricted graph has at most nk vertices, so Stage 2 and the bounds of Section 4.5 apply unchanged. The counting restrictions `useAtMost` j and `useExactly` j are not of this form: an independence constraint forbids pairs of slots, whereas `useAtMost` bounds the number of distinct colors, and `useExactly` is not even preserved when slots are removed from a selection. The checker therefore rejects `useExactly`, and `useAtMost` j with $j < k$, on the exact path (Section 9.2); `useAtMost` j with $j \ge k$ is vacuous. Theorems 4.4 and 4.6 below hold for the restricted graph, with "k-colorable" read as "k-colorable under the restrictions". The robust decoders of Section 4.4 guarantee only properness under restrictions, since a fresh color may lie outside $L(v)$.

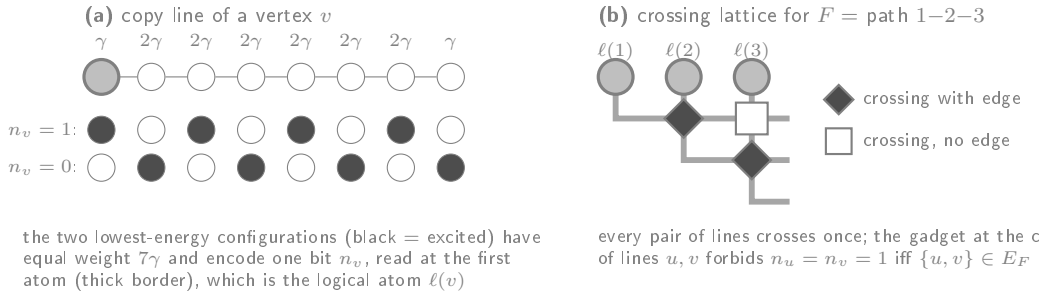


Figure 2: The construction of Theorem 4.4, schematically. (a) A copy line of even length: interior atoms of weight 2γ, end atoms of weight γ, shifted to $\gamma \pm w_F(v)/2$ (first/last) in the weighted case so that $n_v = 1$ gains $w_F(v)$; the two alternating configurations carry one bit. (b) The full crossing lattice: every pair of lines crosses once, at a crossing-with-edge gadget (diamond) if the vertices are adjacent in F and a plain crossing gadget (square) otherwise. Coordinates, gadget internals and weights are those of Nguyen et al. (2023); Figure 3 (c) shows a complete instance.

4.2. Stage 2: independent set → unit-disk MWIS

G'_k is generally not a UDG. Stage 2 embeds it into a weighted UDG with the copy/crossing construction of Nguyen et al. (2023), which builds on the Rydberg MWIS encoding of Pichler et al. (2018). The construction is not a contribution of this paper; we use it as a black box behind the following interface.

Theorem 4.4 (Gadgetized embedding, after Nguyen et al. (2023)). *Let $F = (V_F, E_F)$ be an arbitrary (unweighted) graph. There is a polynomial-time procedure* gadgetize *that maps F to a weighted unit-disk graph $A = (p, r, w)$ on a grid, together with a designated set of* logical *atoms $\ell : V_F \hookrightarrow V(A)$ and a constant $C \geq 0$, such that*

$$\mathrm{MWIS}(A, w) = \alpha(F) + C,$$

and every maximum-weight independent set S_A of A restricts on the logical atoms to a maximum independent set $S = \ell^{-1}(S_A \cap \ell(V_F))$ of F. The number of atoms is at most $4|V_F|^2$ (Nguyen et al., 2023, Sec. V A), so $|V(A)| = O(|V_F|^2)$ in the worst case. We instantiate the theorem only at $F = G'_k$, the color-choice graph of Definition 4.1.

Nguyen et al. (2023) treat a vertex-weighted source; in our notation, their construction gives $\mathrm{MWIS}(A, w) = \mathrm{MWIS}(F, w_F) + C$, and the pipeline uses only $w_F \equiv 1$. The target carries non-uniform weights: 2γ on copy-line atoms, 4γ on the four interior atoms of each crossing gadget, and $\gamma + w_F(v)/2$ and $\gamma - w_F(v)/2$ on the first and last atom of each line, with $2w_F(v) \leq \gamma$.

How the construction works. Two atoms interact only when they are close, so a long edge cannot be realized directly. Each vertex $v \in V_F$ becomes a *copy line*, a chain of atoms whose two lowest-energy configurations are the two alternating patterns, so that the chain carries one bit n_v, readable at its first atom, the logical atom $\ell(v)$. Each line is drawn as a vertical and a horizontal segment, and the lines form an upper-triangular *crossing lattice*. In the full lattice, which Theorem 4.4 describes, every pair of lines crosses exactly once. A crossing receives a *crossing gadget*, which lets the two lines pass without coupling, if $\{u, v\} \notin E_F$, and a *crossing-with-edge gadget*, which additionally forbids $n_u = n_v = 1$, otherwise (Figure 2). The full lattice consists of $|V_F|(|V_F| - 1)/2$ tiles of at most eight atoms each, and Nguyen et al. (2023, Sec. V A) bound the total, including the copy-line atoms on the boundary of the lattice, by $4|V_F|^2$. The lattice diagonal sets the blockade radius, and the constant C absorbs the fixed contribution of the ancillas. If the lines are ordered so that each ends after meeting its last neighbor (a path decomposition of F), pairs of lines that no longer meet need no crossing; this and local simplification rules shrink the lattice on sparse inputs (Nguyen et al., 2023, App. A) (Sections 4.5 and 9.5). We take the gadgets and weights verbatim from Nguyen et al. (2023).

A combinatorial gadget. Our implementation also builds a simpler, non-geometric instance that satisfies the same interface except for the unit-disk property. It is the weighted form of Poljak's reduction Poljak (1974), which subdivides every edge twice and raises the independence number by exactly the number of edges.

Lemma 4.5 (Edge-path gadget). *Let $F = (V_F, E_F)$ be a graph, and let $A_4(F)$ be obtained from F by replacing every edge $\{u, v\}$ by a path $u - a_{uv} - b_{uv} - v$ through two new vertices, with weight 1 on V_F and weight 2 on every new vertex. Then $\mathrm{MWIS}(A_4(F)) = \alpha(F) + 2|E_F|$, and every maximum-weight independent set of $A_4(F)$ restricts on V_F to a maximum independent set of F. The instance has $|V_F| + 2|E_F|$ vertices.*

Proof. Let S_A be an independent set of $A_4(F)$, let $S = S_A \cap V_F$, and let $e(S)$ be the number of edges of F with both ends in S. On each edge path at most one of a_{uv}, b_{uv} is in S_A, and none is when $u, v \in S$; hence $w(S_A) \le |S| + 2(|E_F| - e(S))$. Deleting one end of every edge inside S leaves an independent set S' of F with $|S'| \ge |S| - e(S)$, so $w(S_A) \le |S'| + 2|E_F| - e(S) \le \alpha(F) + 2|E_F| - e(S)$. Conversely, for a maximum independent set I of F, choosing on every edge path the new vertex not adjacent to an end in I (at most one end is in I) gives an independent set of weight $\alpha(F) + 2|E_F|$. So $\mathrm{MWIS}(A_4(F)) = \alpha(F) + 2|E_F|$, and at an optimum the bounds are tight: $e(S) = 0$ and $|S| \ge \alpha(F)$, so S is a maximum independent set of F.

The type rule S2 of Section 8.3 records this offset $C = 2|E_F|$. In general, $A_4(F)$ is *not* a unit-disk graph: a vertex of degree d in F is the center of an induced star $K_{1,d}$ in $A_4(F)$, which is impossible for $d \ge 6$ (Proposition 5.4). The metric construction of Theorem 4.4 is therefore what a device runs; $A_4(F)$ is a checkable stand-in used by the implementation and the direct simulations of Section 9.5.

4.3. Decoding and end-to-end correctness

Decoding composes the two inverses. Given a measured independent set S_A of A, restrict it to the logical atoms and apply ℓ^{-1} to obtain $S \subseteq V'$. The readout is *accepted* only if S holds exactly one slot (v, i) for every $v \in V$ and the map $c(v) = i$ is proper on every edge of G (and, for a restricted instance, satisfies the restrictions of Proposition 4.3), an $O(nk+m)$ check; otherwise the shot is rejected, or repaired by the robust decoders of Section 4.4. A count of n logical atoms is not enough, because the gadget enforces the edges of G'_k only at an optimum: for K_2 at $k = 1$ the gadget instance is the path $u - a - b - v$ with weights $1, 2, 2, 1$, and its suboptimal independent set $\{u, v\}$, of weight $2 < 3$, has $n = 2$ logical atoms yet gives both ends of the edge the same color. Optimal sets always pass (Theorem 4.6 (ii)).

Theorem 4.6 (End-to-end exactness). *Let $A = \mathsf{gadgetize}(G'_k)$ for G'_k the color-choice graph of (G, k). Then:*

(i) $\mathrm{MWIS}(A, w) = n + C$ *if and only if G is k-colorable.*

(ii) Any maximum-weight independent set of A that contains n logical atoms decodes, through ℓ^{-1} followed by the Stage-1 readout, to a proper k-coloring of G.

(iii) The construction and the decoder both run in time polynomial in n and k, and the decoder is an exact inverse of the construction; hence there is no chromatic-number inflation.

Proof. By Theorem 4.4 at $F = G'_k$, $\mathrm{MWIS}(A, w) = \alpha(G'_k) + C$, and by Theorem 4.2, $\alpha(G'_k) \le n$ with equality iff G is k-colorable; this is (i). For (ii), a maximum-weight independent set of A restricts to a maximum independent set S of G'_k, which the Stage-1 bijection reads off as a proper coloring when $|S| = n$. For (iii), Stage 1 takes $O(nk^2 + mk)$ time, matching $|V'| = nk$ and $|E'| = n\binom{k}{2} + mk$; Stage 2 is polynomial by Theorem 4.4; both readouts are linear; and exact inverses compose.

Remark 4.7 (Sources of uncertainty). Theorem 4.6 never assumes that the analog solve succeeds. The two remaining risks are (a) *feasibility*, that is, whether A fits within $N_{\max}$ atoms and area $A_{\max}$ (Section 4.5), and (b) *solve quality*, that is, whether the sweep returns a maximum-weight independent set or a lighter one. Neither is a question about the correctness of the reduction. A returned set can always be verified: a readout that passes the check above certifies a valid coloring, whereas a failed check means only that no

witness was found, not that none exists. A non-colorability verdict therefore always needs an exact classical argument; the decomposition of Section 9.7 supplies one (Theorem 8.5 (ii)), and the device is used only to find colorings. Section 4.4 shows that, on the combinatorial gadget, a suboptimal solve costs at most one extra color per unit of lost weight.

4.4. Robustness to imperfect solves

Every result so far assumes that the device returns an optimum. A sweep that is too fast, or noise, leaves the array in a low-lying excited state instead, so the question is how the decoder handles a near-optimal answer. For the combinatorial gadget of Lemma 4.5 the loss is additive: every unit of weight that the device loses costs at most one extra color. A solution's *gap* is its distance to the optimum: $\mathrm{MWIS}(A) - w(S_A)$ for the gadget instance, $\alpha(G'_k) - |S|$ for G'_k, and the number of colors beyond k for colorings of G.

Lemma 4.8 (Robust edge-path gadget). *Let S_A be an independent set of $A_4(F)$ with $w(S_A) \geq \mathrm{MWIS}(A_4(F)) - \varepsilon$. Deleting one end of every edge of F whose two ends both lie in S_A leaves an independent set S' of F with $|S'| \geq \alpha(F) - \varepsilon$, in time $O(|E_F|)$.*

Proof. With $S = S_A \cap V_F$, the proof of Lemma 4.5 gives $w(S_A) \leq |S'| + 2|E_F| - e(S) \leq |S'| + 2|E_F|$, and $\mathrm{MWIS}(A_4(F)) = \alpha(F) + 2|E_F|$; hence $|S'| \geq w(S_A) - 2|E_F| \geq \alpha(F) - \varepsilon$.

Proposition 4.9 (Robust Stage-1 decoding). *Let S be an independent set of G'_k with $|S| = \alpha(G'_k) - \varepsilon$, and let $d = n - \alpha(G'_k)$, so that $d = 0$ exactly when G is k-colorable (Theorem 4.2). Coloring each vertex v by the index of its selected slot $(v, i) \in S$, and each vertex without a selected slot by a fresh color of its own, produces in time $O(nk)$ a proper coloring of G with at most $k + d + \varepsilon$ colors.*

Proof. The choice cliques allow at most one selected slot per vertex, so the coloring is well defined, and the conflict edges forbid two adjacent selected vertices from sharing a color. A fresh color is used by one vertex only, so it never conflicts. Exactly $n - |S| = d + \varepsilon$ vertices receive fresh colors.

Theorem 4.10 (End-to-end robustness). *Let the device return an independent set S_A of $A_4(G'_k)$ with $w(S_A) \geq \mathrm{MWIS}(A_4(G'_k)) - \varepsilon$. Composing the decoders of Lemma 4.8 and Proposition 4.9 produces in polynomial time a proper coloring of G with at most $k + d + \varepsilon$ colors, so at most $k + \varepsilon$ if G is k-colorable.*

Proof. Lemma 4.8 gives an independent set of G'_k with gap at most ε; Proposition 4.9 decodes it.

The guarantee is *additive*, so a large offset does not weaken it. Cazals, Sorondo, Onofre, Dalyac, Coelho and Vitale (2025) show that reductions to unit-disk MWIS do not preserve approximation *ratios*: because the optimum of the mapped instance is dominated by the offset C, a relative gap on that instance may correspond to an arbitrarily poor solution of the source. Theorem 4.10 gives the complementary positive statement. Absolute gaps transfer one for one, so a solve that loses ε units of weight costs at most ε extra colors, however large C is. Theorem 4.10 is an instance of the graded form of the type system's decoder arrow (Refinement D, Section 8.4); both decoders and the theorem itself, stated on the explicit edge list of G'_k, are mechanized in Rocq (Section 9.1). The metric case is open (Section 11).

4.5. Size and feasibility bounds

Proposition 4.11 (Atom budget). *For an instance (G, k) with $|V| = n$, the gadgetized UDG A uses $nk \leq |V(A)| \leq 4(nk)^2$ atoms, so $|V(A)| = O\big((nk)^2\big)$. The instance is hardware-feasible only if $|V(A)| \leq N_{\max}$ and the layout fits within $A_{\max}$ under spacing $d_{\min}$.*

Proof. Stage 1 produces $|V'| = nk$ vertices, and Theorem 4.4 at $F = G'_k$ gives at most $4|V'|^2$ atoms. The lower bound is the logical-atom count.

Measured against a device of a few hundred atoms Wurtz et al. (2023), the quadratic blow-up is the practical bottleneck. It is not an artifact of this construction: under the Exponential Time Hypothesis, any polynomial reduction from MWIS on arbitrary graphs to MWIS on unit-disk graphs incurs a quadratic blow-up in vertex count Cazals et al. (2025). Two techniques reduce the instance size. *Simplification*:

repeatedly deleting vertices of degree $< k$, which can always be colored last, leaves the k-core, and the k-core is k-colorable iff G is. Splitting at cut vertices into biconnected components is also safe, because the colorings of the components can be permuted to agree on the cut vertices. For $k \leq 2$ no device is needed, since 2-colorability is bipartiteness. *Line ordering*: a copy line need only extend until it has met every line it interacts with, and ordering the lines of F by a path decomposition shrinks the crossing lattice to $O(|V_F|\,\mathrm{pw}(F))$ (Nguyen et al., 2023, App. A). We lift this bound to G'_k.

The *vertex separation* of a vertex order π is the largest number, over the prefixes of π, of prefix vertices with a neighbor outside the prefix; its minimum over all orders is the pathwidth Kinnersley (1992). Write π_k for the order of $V' = V \times \{1, \ldots, k\}$ that lists the k slots of each vertex consecutively, following π.

Proposition 4.12 (Pathwidth bound). *Let π be a vertex order of G.*

(i) If π has vertex separation s, then π_k has vertex separation at most $k(s+1)-1$; hence $\mathrm{pw}(G'_k) \leq k(\mathrm{pw}(G)+1)-1$.

(ii) (Nguyen et al., 2023, App. A) *Let the lines of a graph F be placed in a given order on a lattice of cells of constant size, such that (a) each line is a vertical and a horizontal segment, and its horizontal segment ends in the column of its last neighbor in the order; (b) a line occupies a row of cells only until that column, and freed rows are reused; and (c) every crossing and every turn occupies one cell with a constant number of atoms, and simplification only removes atoms. If the order has vertex separation s', the placement uses $O(|V_F|\,(s'+1))$ atoms.*

Consequently G'_k placed by π_k needs $O\big(nk^2(s+1)\big)$ atoms, linear in n for bounded pathwidth and fixed k.

Proof. (i) The neighbors of a slot (u, c) are the other slots of u and the slots (x, c) with x adjacent to u. Consider a prefix of π_k that ends inside the slots of a vertex v: a prefix slot with a neighbor outside the prefix belongs either to v (at most $k-1$ slots) or to a vertex before v with a neighbor at or after v, and there are at most s such vertices, so at most $ks+k-1$ slots qualify. If the prefix ends after the last slot of v, only slots of the at most s vertices up to v with a neighbor after v qualify, at most ks. The pathwidth statement follows because vertex separation and pathwidth coincide Kinnersley (1992). (ii) Call a line *active* from its placement until its last neighbor has been placed. At most $s'+1$ lines are active at once, so by (a) and (b) the horizontal segments use at most $s'+1$ rows of cells, and every vertical segment spans $O(s'+1)$ cells. By (a), the total horizontal length is the sum, over the cuts of the order, of the lines active across the cut, which is at most $|V_F|\,s'$ cells. Every crossing sits where a horizontal segment meets a vertical one, so there are at most that many, and by (c) each crossing and each turn contributes a constant number of atoms.

Conditions (a)–(c) describe the path-decomposition construction of Nguyen et al. (2023, App. A); Section 9.6 confirms the predicted linear growth on its reference implementation QuEra Computing Inc. (2025). The bound does not contradict the lower bound of Cazals et al. (2025), which concerns arbitrary graphs; the pathwidth of an arbitrary graph can be linear in its order.

4.6. Worked example: 3-coloring a small graph

Example 4.13. Let $G = C_4$ with vertices $v_1v_2v_3v_4$ and $k = 3$ (Figure 3). Stage 1 builds G'_3 with $|V'| = 12$ slots and $|E'| = 4\binom{3}{2} + 4 \cdot 3 = 24$ edges: a choice triangle per vertex and three conflict edges per edge of C_4. The proper coloring $(1, 2, 1, 2)$ corresponds to the independent set $\{(v_1, 1), (v_2, 2), (v_3, 1), (v_4, 2)\}$ of size $n = 4$, so $\alpha(G'_3) = 4$ and C_4 is 3-colorable; at $k = 1$ the same construction gives $\alpha(G'_1) = 2 < 4$, correctly reporting that C_4 is not 1-colorable. Gadgetizing G'_3 gives a weighted UDG of 285 atoms, 12 of them logical; an MWIS restricted to the logical atoms is an independent set of size 4 of G'_3, whose readout is a proper 3-coloring such as $(1, 2, 1, 2)$. No inflation occurs at any step.

5. Secondary path: heuristic geometry and the layout optimizer

Given a prescribed weighted graph B (in this paper, always a gadget instance), the *geometric realization problem* asks for a placement p of $V(B)$ and a radius r such that $\mathrm{UDG}(p, r)$ equals the prescribed adjacency

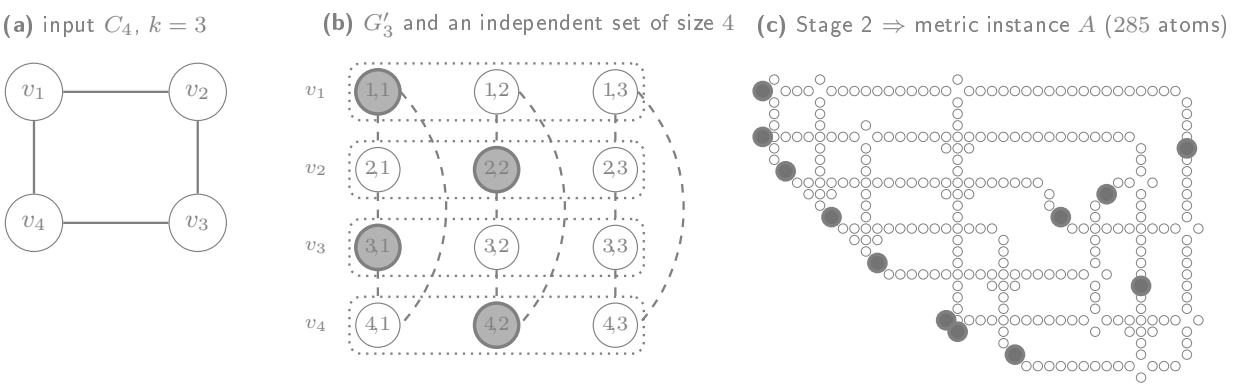


Figure 3: Example 4.13 end to end. (a) The input. (b) G'_3 as a grid of slots (row = vertex, column = color; dotted box = choice clique, edges not drawn; dashed = conflict), with the independent set decoding to $(1, 2, 1, 2)$ shaded. (c) The metric gadget instance produced by the reference implementation QuEra Computing Inc. (2025) under the optimized line order of Section 9.5: 285 atoms on a 27×39 lattice, in which atoms at most one lattice diagonal apart interact; the 12 logical atoms are dark. Its maximum-weight independent sets restrict to those of G'_3 (checked exactly).

$E(B)$, subject to pairwise spacing at least $d_{\min}$, $r \le r_{\max}$, and area at most $A_{\max}$. A *geometric heuristic* runs in polynomial time and may fail to find such a placement; when it fails, it reports infeasibility instead of returning a placement whose realized adjacency differs from $E(B)$. We first show that a geometric supergraph cannot preserve the chromatic number in general, and then give the heuristics and the role they play in Stage 2.

5.1. Why geometric supergraphs cannot bound inflation

Definition 5.1 (Safe supergraph; false edges). A UDG $H = \mathrm{UDG}(p, r)$ on the vertex set V of G is a *safe supergraph* of G if $E(G) \subseteq E(H)$; its *false edges* are $E_{\mathrm{false}} = E(H) \setminus E(G)$.

Proposition 5.2 (Safety ⇒ decoder correctness). *If $E(G) \subseteq E(H)$ and κ is a proper coloring of H, then κ restricted to V is a proper coloring of G. Consequently $\chi(G) \le \chi(H)$.*

Proof. Every edge of G is an edge of H, so the endpoints receive distinct colors under κ. The inequality follows since a proper coloring of H is one of G.

Safe supergraphs therefore never lower χ. The difficulty lies in the other direction.

Lemma 5.3 (Neighborhood packing in a UDG). *For every unit-disk graph $H = \mathrm{UDG}(p, r)$ and every vertex v of H, $\alpha(H[N(v)]) \le 5$.*

Proof. Write $c = p(v)$, and suppose that $x_1, \ldots, x_6$ in the closed disk D of radius r about c are pairwise more than r apart. No x_i equals c, since c is within r of every point of D. Sorting the x_i by angle about c, some two consecutive points a, b subtend an angle $\theta = \angle acb \le 60°$; assume $\|c - a\| \le \|c - b\|$. In the (possibly degenerate) triangle abc the angles at a and b sum to $180° - \theta \ge 120°$, and the angle at a is at least the angle at b, since it faces the longer side; hence it is at least $60° \ge \theta$. As the larger angle faces the larger side, $\|a - b\| \le \|c - b\| \le r$, a contradiction.

Proposition 5.4 (Stars that are unit-disk graphs). *In the plane, the star $K_{1,t}$ is a unit-disk graph if and only if $t \le 5$. Hence the constant 5 in Lemma 5.3 is sharp.*

Proof. For $t \le 5$, place the center at c and t leaves at $c + r(\cos \frac{2\pi j}{5}, \sin \frac{2\pi j}{5})$: every leaf is at distance r from c, and any two leaves are at distance at least $2r \sin(\pi/5) \approx 1.176\, r > r$. For $t \ge 6$, the leaves would form an independent set of size $t > 5$ inside $N(c)$, contradicting Lemma 5.3.

Theorem 5.5 (Supergraph inflation is unbounded). *There is a family of graphs with $\chi = 2$ on which every safe UDG supergraph has $\chi(H) \to \infty$. Concretely, for the star $K_{1,n}$,*

$$\chi(K_{1,n}) = 2, \qquad \chi(H) \;\geq\; \left\lceil \tfrac{n}{5} \right\rceil + 1 \quad \textit{for every safe UDG supergraph } H.$$

Proof. $K_{1,n}$ is bipartite, so $\chi = 2$. Let H be a safe UDG supergraph with center c and leaves L, $|L| = n$. Safety forces every edge $\{c, \ell\}$ into H, so all leaves lie within distance r of $p(c)$, i.e. $L \subseteq N(c)$. In any proper coloring of H, each color class is an independent set of H, hence an independent set within $N(c)$, which by Lemma 5.3 has size ≤ 5. Covering the n leaves therefore needs at least $\lceil n/5 \rceil$ color classes, and c, adjacent to every leaf, one more. The bound is attained: put five groups of at most $\lceil n/5 \rceil$ leaves within $r/20$ of the points at distance $0.95r$ from $p(c)$ in the directions of Proposition 5.4; each group is a clique, distinct groups are more than r apart, and $\chi(H) = \lceil n/5 \rceil + 1$.

5.2. The layout loss and the seven heuristics

For the gadget instance A with prescribed adjacency E_A, the heuristics minimize the *realization* loss

$$L_{\text{layout}}(p, r) = \underbrace{\sum_{\{u,v\} \in E_A} \max(0, d_{uv} - r)^2}_{\text{prescribed edges within } r} + \lambda \sum_{\{u,v\} \notin E_A} \max(0, r + \epsilon - d_{uv})^2 + \mu \sum_{u \neq v} \max(0, d_{\min} - d_{uv})^2, \quad (1)$$

where $d_{uv} = \|p(u) - p(v)\|_2$. The three terms penalize a prescribed edge longer than r, a non-edge shorter than $r + \epsilon$, and a pair of atoms closer than $d_{\min}$. The loss measures how far a placement is from realizing E_A exactly; it does not compare the placement with G. The seven heuristics initialize, run, repair and tune this optimization.

H1 (loss minimization). Gradient descent on (1) over p and r, then a hard feasibility check (every prescribed edge realized, every non-edge excluded, all distances $\geq d_{\min}$); a missing edge triggers $r \leftarrow \max_{\{u,v\} \in E_A} d_{uv}$ and a rebuild. Each step costs $O(|V(A)|^2)$. **H2 (chain-aware weighting).** Weight the loss terms by gadget-chain length, so that edges routed through many crossings are compacted first. This weighting, a geometric analogue of DSATUR's saturation degree Brélaz (1979), targets the $O((nk)^2)$ bottleneck. **H3 (over-packing control).** Every UDG clique lies in some $N[v]$; detect spurious dense clusters locally in $O(|V(A)|\Delta^2)$ and push the offending atoms apart. The hardware bounds clique size: let the *device clique bound* $\omega_{\max}$ be the largest clique number of any UDG(p, r) with $r \leq r_{\max}$ and pairwise spacing at least $d_{\min}$. This is a device constant, not the clique number of a particular graph.

Proposition 5.6 (Hardware-bounded cliques). *With $\omega_{\max}$ as just defined,*

$$\omega_{\max} \;\leq\; \left\lfloor \left(1 + \tfrac{2 r_{\max}}{\sqrt{3}\, d_{\min}}\right)^2 \right\rfloor.$$

Proof. A clique K of a realizable UDG(p, r) has diameter at most r, so by Jung's theorem Jung (1901) it lies in a closed disk of radius $R = r/\sqrt{3}$. The open disks of radius $d_{\min}/2$ around its atoms are pairwise disjoint and lie in the concentric disk of radius $R + d_{\min}/2$, so comparing areas, $|K|\,(d_{\min}/2)^2 \leq (R + d_{\min}/2)^2$, i.e. $|K| \leq \left(1 + 2r/(\sqrt{3}\, d_{\min})\right)^2$. Use $r \leq r_{\max}$ and integrality.

The bound caps local density in terms of physical parameters alone Henriet et al. (2020); Scholl et al. (2021); the optimal planar packing density Fejes Tóth (1953) would improve the constant asymptotically from $4/3$ to $2\pi/(3\sqrt{3}) \approx 1.21$, which we do not need.

H4 (Laplacian initialization). Start H1 from the Fiedler vector and the next Laplacian eigenvector Fiedler (1973); Chung (1997); Belkin and Niyogi (2001), $p_0(v_i) = (\phi_2(i), \phi_3(i))$, which places adjacent atoms close together Fruchterman and Reingold (1991); $O(|V(A)|^2)$ dense, $O(t\,|E_A|)$ for t sparse iterations. **H5 (monotone local repair).** Push or pull offending pairs, accepting a move only if it strictly decreases the potential $\Phi = \sum_{\{u,v\} \in \mathcal{X}} \eta_{uv}$ over the violations $\mathcal{X}$, with integer weights $\eta_{uv} \geq 1$. As Φ is a nonnegative

integer that every accepted move decreases, at most $\Phi_0 \le |V(A)|^2 \max \eta$ moves occur. **H6 (radius sweep).** With p fixed, sort the pairwise distances; the minimum feasible radius is $r^\star = \max_{\{u,v\}\in E_A} d_{uv}$, and over $[r^\star, r_{\max}]$ we select a Pareto point in (realized non-edges, area, r) Miettinen (1999); $O(|V(A)|^2 \log |V(A)|)$, plus $O(|V(A)|^2)$ per radius. **H7 (quality as evaluation).** Discrete measures such as realized non-edges and decoded gap are piecewise constant in (p, r) and give no useful gradient, so they are used only to *evaluate* and *select* layouts.

5.3. Why the heuristics are correctness-neutral

Used as a *stand-alone supergraph backend*, the heuristics would place the vertices of G itself, color $H = \mathrm{UDG}(p, r)$, and restrict. By Proposition 5.2 this never lowers χ, but by Theorem 5.5 nothing bounds how far it raises it; we retain this backend only for comparison (Appendix A lists the families on which it is exact). Inside Stage 2, by contrast, the gadget instance fixes the adjacency and the weights exactly (Theorem 4.4); the heuristics only realize this adjacency, minimizing atoms, crossings and chain length under the hardware constraints. In this role the heuristics cannot affect correctness: a poor layout can make the instance infeasible or make the ground state harder to reach, but it cannot make a decoded answer wrong. For a lattice construction such as that of Theorem 4.4, whose coordinates and radius are fixed once the copy lines are ordered, the optimizer acts through that order (H4 initializes it, an H5-style repair improves it, H7 selects by atom count; Section 9.5), while H1–H3 and H6 apply to free-form gadget layouts.

6. Integrated pipeline and complexity

The compiler chains the pieces of Figure 1. It builds G'_k in $O(nk^2+mk)$ time (Theorem 4.2) and gadgetizes it in polynomial time (Theorem 4.4), checking $N = |V(A)| \le N_{\max}$, where $N = O((nk)^2)$ (Proposition 4.11). It then lays out A: H4 initializes, H1–H2 run for a budget of $T = \mathrm{poly}(N)$ steps followed by the feasibility check (raising r or rebuilding on failure), H3 and H5 repair, and H6 selects $(p, r^\star)$ with $r^\star \le r_{\max}$ and area at most $A_{\max}$, at the costs of Section 5.2. Finally, the device solves MWIS on A at $(p, r^\star)$. The decoder returns the Stage-1 readout of $S = \ell^{-1}(S_A)$ in $O(N)$ time if this readout passes the check of Section 4.3, and reports a solve failure otherwise (Theorem 4.6). The layout can affect feasibility, never correctness. The compiler also never has to solve UDG *recognition*, which is NP-hard Breu and Kirkpatrick (1998), because it only realizes a prescribed adjacency.

Corollary 6.1 (Polynomial compilation). *With a fixed iteration budget $T = \mathrm{poly}(N)$, the compiler runs in time polynomial in n and k for everything except the analog solve, and emits a hardware instance of $O((nk)^2)$ atoms together with an exact decoder.*

Proof. Each step is polynomial in $N = O((nk)^2)$ and T: $O(TN^2)$ for H1–H2, $O(N\Delta^2)$ per H3 pass, at most $\Phi_0 = O(N^2 \max \eta)$ H5 moves, and $O(N^2 \log N)$ for H6. Exactness of the decoder is Theorem 4.6.

7. Related work

Graph coloring and unit-disk graphs. Deciding k-colorability is NP-complete for $k \ge 3$ Karp (1972); Garey and Johnson (1979), and χ is hard even to approximate Zuckerman (2007). It has been studied through the chromatic polynomial Tutte (1954) and topological methods Lovász (1978); the Mycielskians Mycielski (1955) of our benchmark (Section 9.7) are the standard triangle-free graphs of large χ. Practical colorings come from greedy orderings Welsh and Powell (1967) and DSATUR Brélaz (1979); the clique number gives the lower bound $\omega(G) \le \chi(G)$ West (2001) and is itself NP-hard to compute Bomze, Budinich, Pardalos and Pelillo (1999). UDGs are the intersection graphs of unit disks Clark, Colbourn and Johnson (1990). Recognizing them is NP-hard Breu and Kirkpatrick (1998); Kang and Müller (2012), which is why we never try to realize the input graph exactly. On UDGs themselves, coloring and maximum independent set admit simple constant-factor approximation heuristics Marathe, Breu, Hunt, Ravi and Rosenkrantz (1995). Distance-based adjacency also constrains colorings: some unit-distance graphs already have $\chi = 4$ Moser and Moser (1961). Our two packing bounds (Lemma 5.3, Proposition 5.6) are proved in full; the planar packing density Fejes Tóth (1953) would only sharpen a constant.

Rydberg MWIS and gadgetized reductions. The identification of the blockade ground state with UDG–MWIS is due to Pichler et al. (2018), demonstrated on hundreds of atoms by Ebadi et al. (2022) and with site-resolved weights by de Oliveira et al. (2025). Nguyen et al. (2023) introduced the copy and crossing gadgets that encode MWIS on an *arbitrary* graph as MWIS on a weighted lattice UDG with at most quadratic overhead, a known offset and a structural decoder; ancilla chains acting as "quantum wires" serve a similar purpose Kim, Kim, Hwang, Moon and Ahn (2022). Cazals et al. (2025) prove the ETH-conditional quadratic lower bound for such reductions, the counterpart of our size wall, and propose a linear-overhead divide-and-conquer heuristic. Our primary path composes the classical colorability→MIS reduction with Nguyen et al. (2023). We take the atom budget, field and spacing of QuEra's Aquila Wurtz et al. (2023) as reference device constraints, but we have not run the gadget instances on it (Section 10). Angkhanawin, Deger, Pritchard and Adams (2026) instead color a natively embedded UDG directly, using several Rydberg levels per atom as colors (shown numerically for $k = 3$); this needs multi-level control and a unit-disk input, whereas we target the two-level encoding and arbitrary graphs. The platform has been used both to study quantum phases on 256 atoms Ebadi, Wang, Levine, Keesling, Semeghini, Omran, Bluvstein, Samajdar, Pichler, Ho, Choi, Sachdev, Greiner, Vuletić and Lukin (2021) and as a gate-model processor Graham et al. (2022). Photonic devices have shown a sampling advantage Madsen, Laudenbach, Askarani, Rortais, Vincent, Bulmer, Miatto, Neuhaus, Helt, Collins, Lita, Gerrits, Nam, Vaidya, Menotti, Dhand, Vernon, Quesada and Lavoie (2022); variational algorithms are surveyed in Cerezo, Arrasmith, Babbush, Benjamin, Endo, Fujii, McClean, Mitarai, Yuan, Cincio and Coles (2021) and the near-term setting in Preskill (2018).

Layout and decomposition. The layout optimizer uses standard tools in a *correctness-neutral* role: Laplacian eigenvectors Chung (1997); Fiedler (1973) and eigenmaps Belkin and Niyogi (2001), force-directed drawing Fruchterman and Reingold (1991), multidimensional scaling Kruskal (1964), simulated annealing Kirkpatrick, Gelatt and Vecchi (1983) and tabu search Glover and Laguna (1997). Chordal graphs admit clique separators Dirac (1961), which Tarjan's decomposition Tarjan (1985) turns into a divide-and-conquer schedule; for clique separators our Stitch rule needs no boundary search. Tree-width and graph minors Robertson and Seymour (1986) explain why coloring is tractable at bounded width Arnborg and Proskurowski (1989), and decompositions of fixed width can be found in linear time Bodlaender (1996). Our separator search is exhaustive: it tries vertex sets of increasing size, up to four, and among the separating sets of the smallest size keeps one that minimizes the largest remaining component. What is new here is that the device budget $4(wk)^2 \le N_{\max}$ appears as a *typed* premise (Section 8.4).

Reductions with backward maps. A pair of a forward map on instances and a backward map on solutions is a morphism of Blass's category of problems Blass (1995) and the data of a Weihrauch reduction in computable analysis Brattka and Gherardi (2011); our certified reductions add that the backward map is total on optima. L-reductions Papadimitriou and Yannakakis (1991) and the other approximation-preserving reductions Crescenzi (1997) constrain how the backward map transfers solution quality, which our graded decoders record as types (Refinement D). Reductions have been mechanized at scale for undecidability Forster, Larchey-Wendling, Dudenhefner, Heiter, Kirst, Kunze, Smolka, Spies, Wehr and Wuttke (2020) and for NP-completeness, including the Cook–Levin theorem Gäher and Kunze (2021); our development is much smaller and specific to reductions whose decoders are certified on optima.

Types, effects and verified compilation. Our calculus belongs to the line of type-and-effect systems that includes the polymorphic effects of Lucassen and Gifford Lucassen and Gifford (1988), their development into a general discipline Talpin and Jouvelot (1992); Nielson and Nielson (1999), and applications to resource usage Igarashi and Kobayashi (2002) and information flow Volpano, Irvine and Smith (1996). Our decoder grades resemble graded effects and types Katsumata (2014); Gaboardi, Katsumata, Orchard, Breuvart and Uustalu (2016); Orchard, Liepelt and Eades III (2019). Most effect systems approximate what a program does when it runs; ours instead record guarantees about answer-preserving reductions. The safety effect accumulates bounds on chromatic inflation, and the decoder grade tracks how solution gaps propagate through the stages. Verified compilers such as CompCert Leroy (2009) and, for quantum circuits, VOQC Hietala, Rand, Hung, Wu and Hicks (2021) are proved correct once for all inputs. Translation validation Pnueli, Siegel and Singerman (1998) and certifying algorithms McConnell, Mehlhorn, Näher and

```
Decl          ::= "graph" Id "=" GraphExpr | "palette" Id "=" PaletteExpr
                | "restrictions" Id "=" "{" { Restriction ";" } "}"
                | "transform" Id "=" TransformExpr
                | "instance" Id "=" InstanceExpr | "compute" Id "=" QueryExpr
InstanceExpr  ::= "color" GraphRef [ "using" PaletteRef ]
                  [ "subjectTo" RestrRef ] [ "via" TransformRef ]
TransformExpr ::= Stage { ">>>" Stage }
Stage         ::= "colorChoice" Nat | "gadgetize" | "layout" [ Id ] | "emit"
                | "toUDG" "(" TransformMode { "," TransformOption } ")"
                | "superUDG" | "compose" "[" TransformRefList "]" | TransformRef
TransformMode ::= "exact" | "boundedChi" Property
Property      ::= "kColorable" "(" Nat ")" | "chromatic" | "bipartite"
Restriction   ::= "precolor" Vertex ":" Color | "allow" Vertex ":" ColorList
                | "forbid" Vertex ":" ColorList | "sameColor" "(" V "," V ")"
                | "differentColor" "(" V "," V ")" | "useAtMost" Nat
                | "useExactly" Nat | "distinctOn" VertexList
QueryExpr     ::= "chromaticNumber" "(" GraphRef ")"
```

Figure 4: Core grammar of the coloring-specification language, as accepted by the prototype parser of Section 9.2 (elided: literals, transform options, the `encodeColoring` shorthand, optional `Id :: Type` signatures, lexical classes).

Schweitzer (2011) instead check each output, as UNSAT-proof checkers such as DRAT-trim Wetzler, Heule and Hunt (2014) do; some of these checkers are themselves formally verified Tan, Heule and Myreen (2021). We combine the two approaches: the reduction is proved once, and each verdict is checked.

8. The coloring language and its certified-reduction type system

The pipeline is the denotation of programs in a small coloring-specification language. A type system enforces its correctness, and the types of this system describe *problem reductions* rather than data shapes. The object being typed is the classical decoder-carrying reduction of Section 2; the tot-opt judgment below does not define a new notion of reduction. Typing adds three consequences that the classical notion does not give by itself. First, decoders compose together with their total-on-optima certificates, so end-to-end soundness is obtained as a derivation rather than as a separate proof. Second, transforms are *graded* by their effect on the chromatic number and on solution gaps, so a transform that may inflate χ cannot be used unnoticed where a transform that does not inflate it is required. Third, the legal order of stages follows from typing (Corollary 8.4). Base types state that a program is well-formed, whereas problem-types state that a reduction is faithful. We build only the problem-types and leave base types to a standard typed core.

8.1. The surface language

A program declares a `graph`; a `palette` of colors; optional `restrictions` (precolorings, allow/forbid lists, equality constraints, color budgets); a `transform`, which is either a pipeline of stages composed with `>>` or a `toUDG` transform into a unit-disk graph; an `instance` that ties these together; and a `compute` query (Figures 4 and 5). Restrictions on single vertices and on pairs are compiled into the color-choice graph (Proposition 4.3); the counting restrictions `useAtMost` and `useExactly` are rejected on the exact path. The two transform modes correspond to the two notions of transformation in this paper: `toUDG(exact)` requires a faithful unit-disk realization, while `toUDG(boundedChi` P`)` is the supergraph path. Because a UDG supergraph on the same vertices cannot preserve χ in general (Theorem 5.5), `boundedChi` promises only that P is not lowered, together with a bound on how far it rises when such a bound is certified. The safety effects of Section 8.4 grade this promise, and the type system rejects stronger annotations instead of merely discouraging them.

8.2. Problem-types and the certified-reduction judgment

A *problem-type* (sort) names a class of instances together with the indices needed to state preservation:

A, B ::=	$\mathsf{Col}\langle n, k\rangle$	k-coloring instance, $n = \lvert V\rvert$
	$\mathsf{Col}\langle n, k \mid \beta\colon B\rangle$	boundary-precolored (β proper on $B \subseteq V$)
	$\mathsf{IS}\langle N\rangle\{P\}$	independent-set instance, N nodes, refinement P

```
graph gInput = graph { vertices=[1,2,3,4,5],
                       edges=[(1,2),(2,3),(3,1),(1,4),(3,5)] }
palette pNamed = colors [red, blue, green]
restrictions rMain = { precolor 1 : red; differentColor(1,2); useAtMost 3 }
transform tExact   = colorChoice 3 >>> gadgetize >>> layout >>> emit
transform tBounded = toUDG(boundedChi chromatic, keep=[1,2,3], trackOrigin=true)
instance inst1 = color gInput using pNamed subjectTo rMain via tExact
instance inst2 = color gInput using pNamed subjectTo rMain via tBounded
compute  chiInput = chromaticNumber(gInput)
```

Figure 5: A well-typed program (`src/examples/paper_figure.udgc`). `inst1` colors `gInput` with a 3-palette under `rMain` through the exact pipeline `tExact`, typed $\mathsf{Col} \rightsquigarrow \mathsf{Dev}$; `inst2` uses the supergraph transform `tBounded`, typed $\mathsf{Col} \rightsquigarrow \mathsf{Col}$ with effect mono. The prototype checker accepts it with no errors.

$\mathsf{WIS}\langle N,\, C\rangle$	weighted IS, N atoms, additive offset C
$\mathsf{UDG}\langle N,\, r,\, d\rangle$	*exactly* realized unit-disk inst. (radius r, spacing d)
$\widetilde{\mathsf{UDG}}\langle N,\, r,\, d \mid f\rangle$	*approximately* realized, f residual false edges
$\mathsf{Dev}\langle N\rangle \quad (N \le N_{\max})$	device-feasible instance

Each sort A carries a solution domain $\mathrm{Sol}(A)$ and a set $\mathrm{opt}(A)$ of optimal solutions. The solutions of Col are the k-colorings together with the answer "not k-colorable"; a coloring is optimal when it is proper, and the answer is optimal when no proper k-coloring exists. The optima of IS and WIS are their maximum (weight) independent sets in the standard sense. Stage-1 exactness is recorded in the refinement on IS; the canonical refinement is $P_{\mathrm{s1}} \equiv \big(\alpha = n \iff G \text{ is } k\text{-colorable}\big) \wedge \alpha \le n$. The two unit-disk sorts separate exact from approximate layouts. UDG is a placement whose realized graph equals the prescribed gadget adjacency exactly. A layout with false edges realizes a *different* graph and receives the sort $\widetilde{\mathsf{UDG}}$, which is not a legal source for the exact decoder; the types therefore forbid reading an answer off such a layout.

Definition 8.1 (Decoder, total on optima). A *decoder* from B to A is a partial map $\delta : \mathrm{Sol}(B) \rightharpoonup \mathrm{Sol}(A)$. It is *total on optima*, $\mathsf{tot\text{-}opt}(\delta)$, if it maps every optimal solution of B to an optimal solution of A.

Definition 8.2 (Certified reduction). The judgment $\vdash \tau : A \overset{\delta}{\rightsquigarrow} B$ asserts that τ takes an instance of A to an instance of B, that δ is a decoder $B \rightharpoonup A$ with $\mathsf{tot\text{-}opt}(\delta)$, and that any optimum $s^\star$ of B gives the answer to A as $\delta(s^\star)$. Operationally: *solve B on the device, apply δ, obtain the answer to A.*

The rest of the calculus is built on this arrow: each stage rule introduces one arrow, composition chains arrows, and the refinements add indices (grade, effect, budget, slack).

8.3. Per-stage typing rules and soundness

The premises of the rules are *proof obligations*, discharged once and for all by the theorems of Section 4. Stage 1 is the color-choice reduction and Stage 2 the gadget embedding with its static offset C; Stage 3 branches on whether the layout realizes the adjacency exactly, and Fit moves the instance onto the hardware.

$$\frac{n = |V(G)| \quad k \ge 1}{\vdash \mathsf{colorChoice}_k : \mathsf{Col}\langle n, k\rangle \overset{\mathsf{decodeColoring}}{\rightsquigarrow} \mathsf{IS}\langle nk\rangle\{P_{\mathrm{s1}}\}} \;\text{S1}$$

$$\frac{C = 2\,|E(G'_k)| \quad N_A = |V(A)|}{\vdash \mathsf{gadgetize} : \mathsf{IS}\langle N\rangle\{P\} \overset{\mathsf{decodeLogical}}{\rightsquigarrow} \mathsf{WIS}\langle N_A,\ C\rangle} \;\text{S2}$$

$$\frac{\mathsf{realizes}(p, r) \quad \mathsf{spacing}(p) \ge d}{\vdash \mathsf{layout}(p, r) : \mathsf{WIS}\langle N, C\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{UDG}\langle N, r, d\rangle} \;\text{S3-exact}$$

$$\frac{\mathsf{miss}(p, r) = \varnothing \quad f = |\mathsf{false}(p, r)| > 0}{\vdash \mathsf{layout}(p, r) : \mathsf{WIS}\langle N, C\rangle \rightsquigarrow \widetilde{\mathsf{UDG}}\langle N, r, d \mid f\rangle} \;\text{S3-approx}$$

$$\frac{N \le N_{\max}}{\vdash \mathsf{emit} : \mathsf{UDG}\langle N, r, d\rangle \overset{\mathrm{id}}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle}\ \textsc{Fit} \qquad \frac{\vdash \tau_1 : A \overset{\delta_1}{\rightsquigarrow} B \quad \vdash \tau_2 : B \overset{\delta_2}{\rightsquigarrow} C}{\vdash \tau_2 \circ \tau_1 : A \overset{\delta_1 \circ \delta_2}{\rightsquigarrow} C}\ \textsc{Comp}$$

Rule S2 records the offset C and the atom count N_A of the gadget instance A, which Fit compares with $N_{\max}$. The premises shown are those of the combinatorial gadget of Lemma 4.5, which the implementation elaborates, with $N_A = N + C$; for the metric construction of Theorem 4.4, C and N_A are that construction's own, and nothing else changes. S3-exact carries the identity decoder (geometry never relabels a solution); S3-approx carries *no* certified decoder. The only exit from $\widetilde{\mathsf{UDG}}$ is a metric Repair step ($\widetilde{\mathsf{UDG}} \overset{\mathsf{decodeRoute}}{\rightsquigarrow} \mathsf{WIS}$) that re-routes false edges through copy/crossing gadgets, after which S3-exact may be retried.

Theorem 8.3 (Pipeline soundness). *If* $\vdash \Pi : \mathsf{Col}\langle n, k\rangle \overset{\delta}{\rightsquigarrow} \mathsf{Dev}\langle N\rangle$ *is derivable (so that* $N \le N_{\max}$ *and every layer is realized exactly), then:*

(i) for every device measurement m*, if* $\delta(m)$ *is a coloring, it is a proper* k*-coloring of* G*;*

(ii) for every optimal measurement m*,* $\delta(m)$ *is a proper* k*-coloring of* G *if* G *is* k*-colorable, and the answer "not* k*-colorable" otherwise.*

Proof sketch. (i) The Stage-1 decoder δ_{s1} ends with the readout check of Section 4.3, which returns a coloring only if it is proper. (ii) By induction on the derivation, using Comp: each arrow is answer-preserving with a tot-opt decoder, and tot-opt decoders compose, so $\delta = \delta_{\mathrm{s1}} \circ \delta_{\mathrm{s2}} \circ \mathrm{id} \circ \mathrm{id}$ maps an optimum bitstring back through WIS (strip C) to a maximum independent set of G'_k; the refinement P_{s1} says that it has size n if and only if G is k-colorable. The only non-identity geometric step, S3-exact, has premise $\mathsf{realizes}(p, r)$ guaranteeing that the realized graph is A, so their optima coincide. The argument is the meta-theorem of Theorem 4.6, here read off the derivation.

Only part (ii) assumes optimality, and optimality cannot be checked on a device. The implementation therefore relies on part (i) alone: an accepted readout is a certified coloring, and a rejected readout is never reported as non-colorability (Remark 4.7).

Corollary 8.4 (Ill-ordered pipelines do not type). *The three stage-ordering errors are rejected by the types alone, without a separate analysis:* S2 *(*gadgetize before colorChoice*) expects* IS *but is given* Col*;* Fit *(*emit before layout*) expects* UDG *but is given* WIS*; and* reading an answer off a false-edged layout *has no* tot-opt *decoder out of* $\widetilde{\mathsf{UDG}}$ *(only* Repair*, which goes back to* WIS*). The device sort can be reached only through* UDG*. A measured bitstring therefore reaches a coloring only through a chain of* tot-opt *decoders that ends with the readout check, so every accepted coloring is proper (Theorem 8.3 (i)).*

8.4. Four refinements

Refinement A: certificate grade. A grade $g \in \{\mathsf{w}, \chi\}$ on coloring conclusions, with $\mathsf{w} \sqsubseteq \chi$, distinguishes a cheap witness-relative certificate ($\mathsf{Col}^{\mathsf{w}}$: a carried coloring κ proves $\chi(G) \le |\kappa|$, the default) from an exact one (Col^{χ}: $\chi(G) = k$ with a discharged lower bound). All stage rules preserve the grade; only an explicit prove_{χ} step lifts $\mathsf{w} \Rightarrow \chi$. Theorem 8.3 holds at grade w, so sound compilation does not require knowing $\chi(G)$.

Refinement B: the safety effect. The supergraph path rewrites G on the same vertices by adding edges (decoded by restriction), so such a transform $G \mapsto H$ carries a *safety effect* $s \in \{\mathsf{safe}, \mathsf{asafe}(c), \mathsf{mono}\}$: safe means $\chi(H) = \chi(G)$; $\mathsf{asafe}(c)$ means $\chi(G) \le \chi(H) \le \chi(G) + c$; and mono means $\chi(G) \le \chi(H)$. Each is introduced only by a premise that establishes its meaning:

$$\frac{E_{\mathrm{false}} = \varnothing}{s = \mathsf{safe}} \qquad \frac{K \text{ a clique of } G,\ |K| = \ell \quad \kappa \text{ a proper coloring of } H,\ |\kappa(V)| = u}{s = \mathsf{asafe}(u - \ell)} \qquad \frac{E(G) \subseteq E(H)}{s = \mathsf{mono}}$$

The first premise makes preservation trivial; the second bounds the increase by two checkable witnesses, since $\chi(H) \le u$ and $\chi(G) \ge \ell$; the third is Proposition 5.2. For a general supergraph transform it is not known in advance whether it adds any edge, so such a transform is typed mono, or $\mathsf{asafe}(c)$ once a coloring of H and a

clique of G are supplied. Effects form a commutative monoid, $\mathsf{safe} \oplus s = s$, $\mathsf{asafe}(c) \oplus \mathsf{asafe}(c') = \mathsf{asafe}(c{+}c')$ and $\mathsf{mono} \oplus s = \mathsf{mono}$ for $s \neq \mathsf{safe}$, and their meanings compose along $\oplus$, as bounds on increases add.

Because $\oplus$ sums bounds while the intermediate graphs depend on the order in which transforms are applied, two orderings of the same transforms are type-equal only if their accumulated effects and side conditions match. The bound $\sigma(G) \leq 5$ serves as a *screen*, not as a premise of any rule. If $\sigma(G) > 5$, Lemma 5.3 forces a false edge inside an independent set $I \subseteq N(v)$ of size $\sigma(G)$; hence safe is impossible and $\chi(H) \geq 1 + \lceil \sigma(G)/5 \rceil$. This lower bound forces inflation when $\chi(G) \leq \lceil \sigma(G)/5 \rceil$, as for stars, but not for every graph with $\sigma(G) > 5$. The prototype nevertheless rejects the supergraph path when $\sigma(G) > 5$. This rejection is a conservative policy, not a soundness requirement, since mono is sound for every supergraph and $\mathsf{asafe}(c)$ is sound for every discharged witness pair.

Refinement C: decomposition (split/stitch). Decomposition makes the discipline substructural: pieces are resources, and a shared separator is a value the pieces must agree on. A separator S with pieces $G_i = G[S \cup C_i]$ of width $w = \max_i |G_i|$ splits the goal into independently compiled subgoals:

$$\frac{S \text{ separates } G \quad w = \max_i |G_i| \quad 4(wk)^2 \leq N_{\max}}{\vdash \mathsf{split}_S : \mathsf{Col}\langle n, k\rangle \Longrightarrow \{\, \mathsf{Col}\langle |G_i|, k \mid \beta_i{:}S\rangle \,\}_{i=1}^{p}} \textsc{Split} \qquad \frac{\forall i,\ \kappa_i \text{ proper}; \quad \kappa_i|_S = \kappa_j|_S}{\vdash \mathsf{stitch} : \{\kappa_i\}_i \overset{\bigcup}{\rightsquigarrow} \kappa \text{ on } G} \textsc{Stitch}$$

The typed budget $4(wk)^2 \leq N_{\max}$, which is Proposition 4.11 applied to one piece, makes the peak device size depend on the piece width w rather than on n (the analogue of register spilling, with atoms in place of registers). For a clique separator, a color-relabeling coercion discharges the agreement premise without search; in general, discharging it requires a search over the $k^{|S|}$ boundary table. For a 256-atom device the premise admits only $wk \leq 8$.

Theorem 8.5 (Decomposed soundness). *Let* Split *split G at S into pieces $G_i = G[S \cup C_i]$.*

(i) If each piece is compiled by a derivation of Theorem 8.3 and Stitch *applies, then $\kappa = \bigcup_i \kappa_i$ is a proper k-coloring of G, and every device instance has at most $4(wk)^2 \leq N_{\max}$ atoms.*

(ii) If, for every proper k-coloring β of $G[S]$, some piece G_i has no proper k-coloring extending $\beta|_S$, then G is not k-colorable.

Proof. (i) Every edge of G lies inside some G_i, because S separates the components C_i; the κ_i are proper and agree on S, so their union is proper. (ii) A proper k-coloring κ of G would restrict to a proper coloring $\beta = \kappa|_S$ of $G[S]$ and to proper extensions $\kappa|_{G_i}$ of β on every piece, contradicting the hypothesis.

Part (ii) quantifies over every piece and boundary coloring; a device solve returning no coloring does not establish it (Remark 4.7), so Section 9.7 decides each such leaf by an exact classical check.

Refinement D: graded decoders. The $\mathsf{tot\text{-}opt}$ arrow specifies what a decoder does with an optimum; its graded form specifies what the decoder does with any other solution. Give every sort a gap, the distance of a solution from the optimum (Section 4.4), and call a decoder $\delta : \mathrm{Sol}(B) \to \mathrm{Sol}(A)$ *robust with slack transfer* g, a monotone function on $\mathbb{N}$, if $\mathrm{gap}_A(\delta(s)) \leq g(\mathrm{gap}_B(s))$ for every s. Robust decoders compose with their slack transfers:

$$\frac{\vdash \tau_1 : A \overset{\delta_1}{\underset{g_1}{\rightsquigarrow}} B \quad \vdash \tau_2 : B \overset{\delta_2}{\underset{g_2}{\rightsquigarrow}} C}{\vdash \tau_2 \circ \tau_1 : A \overset{\delta_1 \circ \delta_2}{\underset{g_1 \circ g_2}{\rightsquigarrow}} C} \textsc{Comp-g}$$

On the combinatorial path the stage grades are $g_{\mathrm{S2}}(\varepsilon) = \varepsilon$ for the edge-path gadget (Lemma 4.8), $g_{\mathrm{S1}}(\varepsilon) = d + \varepsilon$ with $d = n - \alpha(G'_k)$ for the fresh-color Stage-1 decoder (Proposition 4.9, the gap of a coloring being its number of colors beyond k), and the identity for S3-exact and Fit. Rule Comp-g then derives Theorem 4.10, with $g = \varepsilon \mapsto d + \varepsilon$, for the whole pipeline; Rocq proves this instance as one theorem (Section 9.1). A $\mathsf{tot\text{-}opt}$ decoder is the special case that maps gap 0 to gap 0, so the graded arrow refines

Paper	Rocq (module . name)	Machine-checked statement
sorts; decoder; tot-opt; arrow	`Sorts.Pcol`, `Pis`, `Pwis`, `Pudg`, `Pdev`; `Problem.decoder`, `tot_opt`, `CRed`	definitions
Comp	`Problem.cred_comp`, `comp_tot_opt`, `cred_sound`	certificates compose; a target optimum decodes to a source optimum
S1; Thm. 4.2	`Stages.S1`, `decodeColoring_tot_opt`, `stage1_opt`	a maximum independent selection has size $n \iff k$-colorable
S2	`Stages.gadgetize`, `decodeLogical_tot_opt`, `gadget_mwis`	an optimum has the largest weight, $= n + C \iff$ colorable; C, N indices
S3-exact; Fit	`Stages.layout_exact`, `Stages.emit`	premises `Realizes` and `N <= Nmax` (atom count)
Thm. 8.3	`Pipeline.pipeline_accept_sound`, `pipeline_sound`, `pipeline_decision`	(i) every accepted readout is proper; (ii) an optimal readout decides colorability
Cor. 8.4; S3-approx	`Pipeline.dev_only_via_udg`, `no_gadget_before_choice`, `no_emit_before_layout`, `no_exit_from_udga`	ill-ordered compositions do not type-check (`Fail`); no step derivation
Refinement A	`Effect.Grade`, `gle_refl/trans/antisym`	grade order
Refinement B	`Effect.eplus_id_l/r`, `eplus_assoc`, `eplus_comm`, `respects_comp`, `asafe_intro`, `mono_intro`	commutative monoid; meanings compose along $\oplus$; witnesses give asafe$(u - \ell)$
Lemmas 4.5, 4.8	`Robust.gadget_mwis_value`, `gadget_optimum_restricts`, `gadget_robust`	$\mathrm{MWIS}(A_4(F)) = \alpha(F) + 2\lvert E_F\rvert$; optima restrict to maximum IS; gap ε decodes to gap $\le \varepsilon$
Prop. 4.9; Comp-g	`Robust.stage1_robust`, `robust_comp`	fresh-color decoder proper with excess $n - \lvert S\rvert$; graded decoders compose
Thm. 4.10	`Robust.robust_end_to_end`	gap ε on $A_4(G'_k)$ decodes to a proper coloring with at most $d + \varepsilon$ fresh colors
Refinement C; Thm. 8.5 (ii)	`Certificate.check_coloring_sound`, `check_refutation_sound`	accepted coloring is proper; accepted refutation $\Rightarrow$ no k-coloring

Table 1
Correspondence between the paper and the Rocq development (`proof/` in the artifact; Rocq 9.0.0, axiom-free). The deliberate departures from a one-for-one correspondence are listed in Section 9.1.

the certified-reduction judgment rather than replacing it. The grade $\varepsilon \mapsto \varepsilon$ is the second condition of an L-reduction Papadimitriou and Yannakakis (1991) with $\beta = 1$. Typing adds that the grade travels with the decoder through every stage, so that a stage which does not preserve absolute gaps is visible as a grade not of the form $\varepsilon \mapsto c + \varepsilon$.

9. Implementation, mechanized proofs, and empirical validation

Every numeric result below is reproduced by the scripts in the artifact, and every theorem cited as machine-checked is closed under the global context (no axioms, no `Admitted`).

9.1. Mechanized correctness in Rocq

We formalize the calculus of Section 8 in the Rocq Prover 9.0.0 (formerly Coq), in seven modules that use no external libraries. A *problem type* bundles a solution set with an optimality predicate; a *certified reduction* CRed $A\,B$ carries an executable decoder $B \rightharpoonup A$ together with a total-on-optima certificate. Reductions and their certificates compose, and a single generic soundness lemma is reused at every stage. Table 1 maps each rule and theorem of the paper to its Rocq counterpart.

Scope of the mechanization. The development departs from a one-for-one correspondence in four deliberate ways. (i) *Stage 2 is mechanized at its interface*: a solution is a logical selection paired with an ancilla occupancy, the decoder is the projection, and the offset C and the atom count N are indices of the sort, so `gadget_mwis` does not state $C = 2|E'|$. That value, Lemma 4.5, is proved in the separate `Robust` model

of $A_4(F)$ for an arbitrary edge list, together with its robust form. `Robust` also builds the explicit edge list of G'_k and proves Theorem 4.10 as the single theorem `robust_end_to_end`; this explicit model is not connected to the interface sort of the pipeline, so the exact chain and the robust chain are two results side by side. (ii) *Layout and emit only unwrap solutions*: `Pwis`, `Pudg` and `Pdev` have the same optima but distinct solution types indexed by the atom count (and by C), so composing the stage reductions in a wrong order, or with mismatched atom counts, is a type error in Rocq, recorded by `Fail` commands in `Pipeline`. The geometric and atom-count premises of S3-EXACT and FIT only restrict which pipelines are derivable. (iii) *The choice clique is encoded*: the selection type `Sorts.Sel := nat -> option nat` enforces at most one slot per vertex by construction. The optima of IS are maximum independent selections in the standard sense, and `stage1_opt` proves that a maximum has size n if and only if G is k-colorable. Restrictions (Proposition 4.3) are not mechanized; the property suite checks them (Section 9.2). (iv) *Refinement C is checked through its output*: SPLIT and STITCH are not mechanized as typing rules. Instead, every decomposition run emits a certificate, which the checkers proved sound in `Certificate` accept or reject (Section 9.8). This gives a translation-validation guarantee Pnueli et al. (1998) for each run, not a proof about the procedure. The metric UDG construction of Nguyen et al. (2023) is measured and checked exactly on small instances (Section 9.5), not mechanized.

9.2. The typed front end

The Haskell package `udg-color` (GHC 9.12) implements the surface pipeline: a parser and AST; a checker that tracks each transform's sorts and safety effect, enforces stage ordering by sort adjacency, and applies the atom-budget check of FIT and the $\sigma(G) \leq 5$ gate; an elaborator that lowers well-typed programs, restrictions included (Proposition 4.3), through Stage 1 and the combinatorial gadget; and an emitter that writes the device instance consumed by the simulator. The Haskell decoders are not extracted from Rocq; a property suite checks them against the brute-force oracle. The checker accepts the program of Figure 5 and rejects the stand-alone $\sigma(G) > 5$ supergraph. The prototype covers less than the calculus: it types every `boundedChi` or `superUDG` transform `mono`, since the witnesses that discharge `asafe`(c) exist only after elaboration; it does not carry decoder terms in transform types; it does not implement the grade of Refinement A; and it does not check the problem-type indices, which serve only as annotations.

The type system specifies thirteen ill-typed cases: (1) a palette where a graph is expected; (2, 3, 7) a vertex outside V in a restriction, an edge, or a `keep`/`drop` list; (4) a color outside the palette; (5) `useExactly` exceeding the palette; (6) a restriction set reused on the wrong graph; (8) overlapping `keep`/`drop`; (9) an `exact` transform that drops vertices (S3-EXACT); (10) a weak transform used as a stronger guarantee ($\mathsf{w} \not\sqsupseteq \chi$); (11) a color in a vertex position; and (12) a UDG literal whose points and edges disagree; and (13) a counting restriction (`useExactly`, or `useAtMost` below the palette size) on the exact path. Stage-ordering errors are rejected structurally (Corollary 8.4). The prototype raises codes 1–5, 8, 9 and 13 and the stage-ordering and $\sigma > 5$ rejections; codes 6, 7 and 10–12 are specified but not yet checked. All 15,338 checks pass: a language suite of 4 acceptance and rejection checks, and a property suite of 15,334 oracle checks (Stage-1 sizes, decoder round-trips, gadget offsets, and the restriction encoding of Proposition 4.3 on every graph with at most four vertices).

Slots, vertices and atoms. We keep three counts apart. *Slots* are the nk vertices of G'_k; Sections 9.3 and 9.7 count slots, and the 16-slot ceiling is the simulator's 2^{nk} wall. The $nk + 2|E'|$ *vertices of* $A_4(G'_k)$ are counted only where that instance is simulated directly (Sections 9.4 and 9.5). *Atoms* are the sites of a metric gadget instance (Table 4, Figure 6), and only atoms are compared with a device budget. The FIT check of the prototype therefore reports an error only when the nk logical atoms alone exceed $N_{\max}$, and reports that an instance fits only when the bound $4(nk)^2$ of Proposition 4.11 is within $N_{\max}$; otherwise, fit is not established and has to be decided from the metric layout.

9.3. Validating the reduction by Rydberg simulation

Our ideal-blockade simulator evolves the time-dependent Rydberg Hamiltonian under a QuEra-style adiabatic protocol (ramp the Rabi drive Ω up and down while sweeping Δ from negative to positive); the blockade is imposed as a hard energy penalty on the edges of G'_k. Keeping the most-populated basis states that are independent sets mimics the post-selection of repeated hardware shots. We compare the recovered

n	p	instances	k used	runs colorable / not	oracle = sim	decoded proper
4	0.5	20	1–3	20 / 20	40/40	20/20
5	0.3	20	1–3	20 / 17	37/37	20/20
5	0.5	20	1–3	18 / 20	38/38	18/18
5	0.7	20	1–3	10 / 20	30/30	10/10
6	0.2	20	1, 2	17 / 19	36/36	17/17
6	0.35	20	1, 2	12 / 20	32/32	12/12
7	0.2	20	1, 2	16 / 20	36/36	16/16
8	0.15	20	1, 2	17 / 20	37/37	17/17
total		160		130 / 156	286/286	130/130

Table 2
Random $G(n, p)$ instances against the brute-force oracle (seed 20260901; raw data in `benchmarks/results/random.csv`). "Runs" counts simulated (G, k) pairs at $k = \chi$ (colorable) and $k = \chi - 1$ (not colorable).

|MIS| with a brute-force oracle over the $(k+1)^n$ partial colorings. These runs validate the implementation of Stage 1 and of the decomposition on G'_k; the combinatorial Stage-2 gadget is simulated directly in Section 9.5, and the metric construction is measured in Section 9.5.

We run ten named graphs (P_4, C_4, C_5, K_3, K_4, $K_{2,3}$, $K_{1,4}$, the bull, the wheel W_5 and Petersen), each at $k = \chi$ and $k = \chi - 1$. Petersen ($nk = 30$) exceeds the simulator's wall; all 18 simulated instances match the oracle, and every colorable run decodes to a coloring verified to be proper. Because a small, hand-picked suite risks selection bias, we repeat the comparison on Erdős–Rényi graphs $G(n, p)$ under a protocol fixed before the runs (`benchmarks/run_random.py`). Each cell has 20 instances drawn from a fixed seed, and none is discarded. Each instance is run at every $k \in \{\chi, \chi - 1\}$ with $k \geq 1$ and $nk \leq 16$; one instance for which neither value qualified was run once at $k = \lfloor 16/n \rfloor$, where it is necessarily non-colorable. The densities lie on both sides of the colorability threshold, so that both verdicts occur in every row. Across 160 instances and 286 runs, every run matches the oracle, and all 130 colorable runs decode to colorings verified to be proper (Table 2).

Stage-1 correctness is already established for all graphs and all k by Theorem 4.2, so these 304 runs are *regression checks*. They would detect an implementation defect, including one that a structured suite could miss, but they add no evidence for the reduction itself, which is proved. On the named graphs, the cost of the simulator grows with its 2^{nk} state vector: under one second up to 10 slots, about 3 s at 12, and about 55 s at the 16-slot wall. This wall is a limit of classical simulation, not the device budget $N_{\max}$; the decomposition of Section 9.7 avoids it.

9.4. Imperfect solves

To exercise Theorem 4.10 we make the simulator fail on purpose: the sweep is shortened from 4 to 1.2 time units (40 steps instead of 90–160), so that many shots end in low-lying excited states. We sample 1000 shots from the final state of each run, discard those that violate the blockade (as hardware post-selection would), and decode *every* remaining shot with the robust decoders, including suboptimal ones the exact pipeline would reject. Table 3 covers the named graphs at $k = \chi$ on G'_k, and the combinatorial gadget array $A_4(G'_k)$ on a chosen set of instances within the simulation wall (`benchmarks/results/robust.csv`).

All 14 835 decoded shots satisfy the bound of Theorem 4.10, and every A_4 shot satisfies Lemma 4.8, as the theorems require; this result is again a regression check. The new information is in the other columns. With the short sweep, only 19–85 % of the color-choice shots are optimal, and the exact pipeline would discard the rest. The robust decoder turns every color-choice shot into a proper coloring with at most $k + \varepsilon$ colors, and a greedy completion recovers a proper k-coloring from 67–100 % of these shots. A suboptimal shot is therefore still usable: each unit of lost weight costs at most one extra color, and in most shots it costs none.

9.5. Metric gadget instances and the layout optimizer

We compute the gadget instance A of Theorem 4.4 for G'_k with UnitDiskMapping.jl QuEra Computing Inc. (2025), the reference implementation of Nguyen et al. (2023), in its weighted mode with default

level	instance	k	size	shots	gap 0	gap 1	gap ≥ 2	within bound	k-col. (greedy)
G'_k	P_4	2	8	1000	583	360	57	1000	873
	C_4	2	8	999	576	287	136	999	953
	C_5	3	15	993	187	586	220	993	993
	K_3	3	9	996	424	313	259	996	996
	K_4	4	16	1000	847	115	38	1000	1000
	$K_{2,3}$	2	10	999	490	294	215	999	939
	$K_{1,4}$	2	10	999	319	490	190	999	916
	Bull	3	15	988	208	535	245	988	686
	W_5	3	15	1000	194	433	373	1000	668
A_4	K_2	2	12	956	394	493	69	956	n/a
	K_2	1	4	996	904	89	3	996	n/a
	P_3	1	7	993	533	419	41	993	n/a
	K_3	1	9	967	906	47	14	967	n/a
	P_4	1	10	988	792	173	23	988	n/a
	C_4	1	12	961	438	461	62	961	n/a

Table 3
Deliberately imperfect solves (sweep shortened to 1.2 time units; 1000 shots per instance, blockade-violating shots discarded). "Size" counts the slots of G'_k or the vertices of $A_4(G'_k)$. "Gap" is the distance of a shot from the optimum ($\alpha(G'_k) - |S|$ on G'_k; $\mathrm{MWIS}(A) - w(S_A)$ on A_4). "Within bound" counts shots whose decoded coloring is proper and uses at most $k + d + \varepsilon$ colors (Theorem 4.10); on A_4 it also requires the bound of Lemma 4.8. "k-col. (greedy)" counts shots from which a greedy completion (try the k palette colors before a fresh one) recovers a proper k-coloring; it is not applicable (n/a) on A_4. Instances at $k = 1$ on A_4 are not 1-colorable ($d > 0$).

simplification rules. Since coordinates and radius are fixed by the lattice once the *order* of the copy lines is fixed, the optimizer acts on that order (Section 5.3). We compare three orders: *naive*, the input slot order $(v, c) \mapsto (v-1)k+c$; *optimized*, the H4 order (slots sorted by the Fiedler vector of G'_k) followed by an H5-style repair that accepts an adjacent transposition only if the atom count strictly decreases, so that it terminates (H7: the objective is the measured count, never a surrogate); and *path decomposition*, the library's own order (exact branching for $nk \leq 16$, and greedy with 50 restarts above that size (Nguyen et al., 2023, App. A)), which serves as a strong reference rather than as a baseline. An instance *fits* if $|V(A)| \leq N_{\max} = 256$ and its bounding box lies within the $75 \times 76\,\mu$m field of QuEra's Aquila at its $4\,\mu$m minimum spacing Wurtz et al. (2023); both are necessary conditions. To check that the counted instances are correct, we solved MWIS on A exactly for every instance with $nk \leq 15$ under the naive and optimized orders, and confirmed $\mathrm{MWIS}(A) = \mathrm{MWIS}(G'_k, w) + C$ for one uniform and two random source-weight vectors w; all 24 checks pass (`benchmarks/verify_layout.jl`).

Compared with the naive order, the optimized order reduces the atom count on every instance, by 7–34 %. It also uses fewer atoms than the path-decomposition order on every instance except K_4 at $k = 4$ (688 against 666 atoms). Fewer atoms do not always mean fewer crossings (P_4, C_5). The main limitation, however, is size. Only K_2 and P_4 at $k = 2$ meet the 256-atom budget, and only K_2 also fits the field of view. Even C_4 at $k = 3$ needs 285 atoms; Petersen at $k = 3$ needs about 1.9 thousand and `myciel3` at $k = 4$ about 4.0 thousand. These counts make the quadratic overhead of Proposition 4.11 concrete: on present hardware the exact path is limited to very small instances, and the layout optimizer improves the constant factor but not the growth rate.

Direct simulation of the combinatorial gadget. The runs of Section 9.3 place the blockade on the edges of G'_k and never materialize a Stage-2 instance. We therefore also simulate the combinatorial instance $A_4(G'_k)$ of Lemma 4.5 itself, with the blockade on its edges, on the instances that fit within the classical simulation wall. For the single edge K_2 at $k = 2$ the 12-vertex instance gives $\mathrm{MWIS}(A) = \alpha(G'_k) + C = 2 + 8 = 10$, and its logical vertices decode to the proper coloring $a \to 1$, $b \to 2$; at $k = 1$ the 4-vertex instance gives $\mathrm{MWIS}(A) = 1 + 2 = 3$, whose logical support of size $1 < 2$ correctly reports that K_2 is not 1-colorable.

			naive order			optimized order			path	
Instance	nk	$\lvert A_4\rvert$	atoms	cross.	box	atoms	cross.	box	dec.	fits
K_2, k=2	4	12	29	1	10×11	**19**	1	7×7	29	yes
P_4, k=2	8	28	74	2	11×26	**65**	3	11×23	74	atoms only
C_4, k=3	12	60	314	16	27×43	**285**	15	27×39	314	no
C_5, k=3	15	75	398	19	27×55	**371**	20	26×55	398	no
K_4, k=4	16	112	793	61	51×59	688	50	43×59	**666**	no
Petersen, k=3	30	180	2074	140	63×115	**1867**	131	63×115	1882	no
`myciel3`, k=4	44	336	5506	444	115×171	**4019**	279	83×171	4051	no

Table 4
Metric gadget instances (UnitDiskMapping.jl 0.5.1) under three line orders. "$\lvert A_4\rvert$" is the number $nk + 2\lvert E'\rvert$ of vertices of $A_4(G'_k)$, for comparison; "cross." counts crossing gadgets; "box" is the bounding box in lattice sites; "path dec." is the count under the library's path-decomposition order; bold marks the smallest count per row. "Fits" refers to the best order and requires $\lvert V(A)\rvert \le 256$ and a box within Aquila's field at $4\,\mu$m spacing; "atoms only" means the budget is met but the field is not. Raw data: `benchmarks/results/layout.csv`.

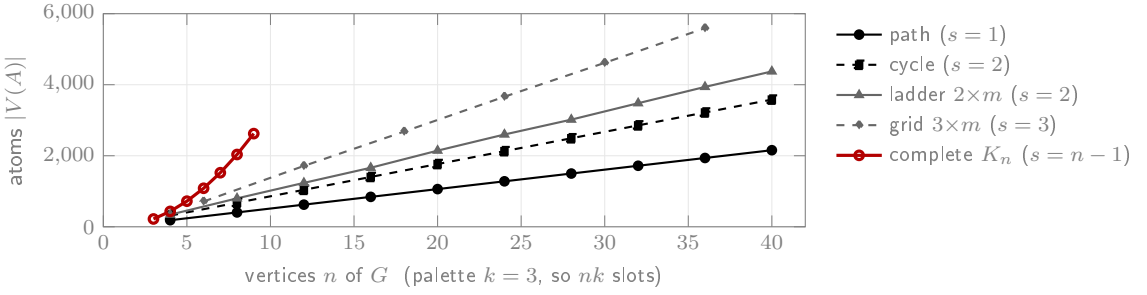


Figure 6: Atoms of the metric gadget instance under the lifted line order (UnitDiskMapping.jl). Bounded-pathwidth families grow linearly in n (Proposition 4.12); complete graphs grow quadratically. The generic bound $4(nk)^2$ is 57 600 at $n = 40$.

These two runs confirm the offset identity on the instance itself, for one colorable and one non-colorable case. Larger instances exceed the wall, and for them the identity rests on Lemma 4.5.

A geometric emulator check. We also ran the `bloqade-analog` emulator, which models the geometric C_6/r^6 interaction instead of an abstract blockade, on a five-atom unit-disk "bowtie" (two triangles sharing an apex); it also returns maximum independent sets. This single unweighted check does not show that the ideal-blockade model matches the hardware; weighted metric instances with per-atom detunings are left for future work (Section 11).

9.6. Scaling with pathwidth

Proposition 4.12 predicts that, under the lifted line order, the atom count of bounded-pathwidth families grows linearly in n. We test this with the reference implementation on paths, cycles, $2\times m$ ladders and $3\times m$ grids with up to 40 vertices at $k = 3$ (so the color-choice graphs have up to 120 vertices), and contrast them with complete graphs K_n, whose pathwidth is $n - 1$ (`benchmarks/run_scaling.jl`).

The measurements agree with the prediction (Figure 6). The atom counts of paths and cycles grow by exactly 54.75 and 90.75 atoms per vertex (a perfect linear fit); those of ladders and $3\times m$ grids grow by about 112 and 162 atoms per vertex, with small residuals; and those of complete graphs grow quadratically. In every case, the vertex separation of the lifted slot order is ks, within the bound $k(s + 1) - 1$. At the largest sizes the instances are 8 to 27 times smaller than $4(nk)^2$, and this ratio grows with n. On the bounded-pathwidth families, the library's greedy path-decomposition order uses fewer atoms than the lifted order only on the two smallest instances (by 4 % on the 2×2 ladder and by 19 % on the 3×2 grid), and up to 24 % more elsewhere. On complete graphs, for which every vertex order has the same separation, the library's order uses 8–23 % fewer atoms throughout. Even with linear scaling, only the smallest of these instances stay

under a 256-atom budget: a path at $k = 3$ needs 185 atoms at $n = 4$ and 404 at $n = 8$. Linear scaling does, however, remove the quadratic growth for sparse structured inputs, and it suits the decomposition below, whose pieces are small and structured.

9.7. Avoiding the simulation wall by certified decomposition

A *separator decomposition*, the classical counterpart of Split and Stitch, avoids the 2^{nk} wall. Given a balanced vertex separator S, we enumerate the proper colorings of S, solve each component plus its boundary as a small piece, and stitch pieces that agree on S; when no separator is available, we pin a free vertex and branch over its k colors. The size of the largest piece depends on the separator width, not on n. Each leaf is first given to the device; a coloring that the device returns is a *witness*, which is checked and then used. If the device returns no coloring, which does not show that none exists (Remark 4.7), an exact search over the leaf's at most 13 slots either finds a coloring (a *device miss*) or proves that none exists. Pinning never changes colorability, so the recursion is exact. A "not k-colorable" verdict is therefore a classical proof, namely the boundary enumeration of Theorem 8.5 (ii) with exact leaf checks. The device supplies colorings, never lower bounds.

Table 5 reports four groups of runs. In the first group, choosing $k > \chi$ raises the whole-instance nk to as much as 192, which would require a monolithic state vector of 2^{192} amplitudes, while every simulated solve stays at ≤ 13 logical slots. This group exercises only the colorable direction, which a palette larger than χ makes easy. The second group runs the same graphs at $k = \chi$ and $k = \chi - 1$, and the third adds further graphs, including the odd cycle C_{21} at $k = 2$; the fourth group, the DIMACS Mycielski graphs, is discussed below. Every verdict agrees with the known chromatic number. The device never missed a colorable leaf: the exact check was needed only on leaves that have no coloring.

The DIMACS Mycielski benchmark. The Mycielskians `myciel3`–`myciel7` of the DIMACS coloring suite Johnson and Trick (1996) are triangle-free graphs from a family whose chromatic number grows without bound; on them the clique lower bound gives only $\chi \geq 2$ ($\omega = 2$). Running `myciel3` in both directions certifies its chromatic number exactly. The run at $k = \chi = 4$ finds a proper coloring on the device. The run at $k = 3$ proves classically that no 3-coloring exists: it exhausts the boundary table, and every leaf on which the device finds no coloring is decided by the exact check. This proof visits 158 leaves in 106 s, including certificate generation; the device colors 75 of the leaves, and the other 83 are certified infeasible. For `myciel4` we run the feasible direction and find an optimal 5-coloring; the infeasible direction, which requires heavy backtracking, exceeds our time budget. In these runs every simulated solve stays at ≤ 11 logical slots, whereas monolithic simulation would need state vectors of 2^{44} and 2^{115} amplitudes. `myciel5` ($n = 47$) and dense instances such as the queen graphs have large separators, which prevent the decomposition from keeping the pieces small; these instances are beyond our reach.

9.8. Checkable certificates in both directions

Theorem 8.5 and the exact leaf checks make the decomposition's verdicts correct; certificates make them *checkable* by a small verified program, independently of the solver, the simulator and the decomposition code. A colorable verdict is certified by the coloring itself. A non-colorable verdict is certified by a *refutation*: a tree over color domains with four kinds of node. $\mathsf{Split}\, v\, c$ branches on whether v gets color c; $\mathsf{Restrict}\, W'$ narrows attention to a sub-piece W'; and the leaves $\mathsf{Clash}\, u\, v$ (an edge whose ends are fixed to the same color) and $\mathsf{Empty}\, v$ (a vertex with no color left) close a branch. A separator node of the decomposition becomes a chain of Splits over the separator; each boundary coloring then either clashes or is followed by a Restrict to the piece that cannot be colored, which is the structure of Theorem 8.5 (ii). Each leaf piece is refuted by Splits down to clashes. The module `Certificate` defines the checker as a Rocq function and proves `check_refutation_sound`: if `check_refutation` $E\ V\ k\ r$ returns `true`, then no map $V \to \{0, \ldots, k-1\}$ is a proper coloring of (V, E); `check_coloring_sound` states the analogous fact for colorings. Both checkers are extracted to OCaml and wrapped in a parser of about 50 lines; the trusted base is the Rocq kernel, the extraction, the OCaml compiler and that parser.

All 26 decomposition runs produce certificates: the 21 rows of Table 5 and five further colorable runs recorded in `decomp.csv` (Frucht at $k = 3$, Heawood and Pappus at $k = 2$, and second runs of Petersen and the dodecahedral graph at $k = 3$). The verified checker accepts all 26, each in under 20 ms. The 19 colorable

Graph	n	m	χ	k	whole nk	max piece	calls	exact	verdict
C_{24} (cycle)	24	24	2	8	192	11	8	0	proper (5 colors)
Ladder 2×10	20	28	2	8	160	11	10	0	proper (4 colors)
Dodecahedral	20	30	3	8	160	13	7	0	proper (7 colors)
Star $K_{1,12}$	13	12	2	10	130	11	12	0	proper (2 colors)
Petersen	10	15	3	8	80	13	3	0	proper (6 colors)
C_{24} (cycle)	24	24	2	2	48	12	4	0	proper
				1	24	7	1	1	**not 1-colorable**
Ladder 2×10	20	28	2	2	40	12	6	0	proper
				1	20	11	1	1	**not 1-colorable**
Dodecahedral	20	30	3	3	60	10	25	9	proper
				2	40	8	16	14	**not 2-colorable**
Star $K_{1,12}$	13	12	2	2	26	3	12	0	proper
				1	13	2	1	1	**not 1-colorable**
Petersen	10	15	3	3	30	10	4	1	proper
				2	20	8	10	8	**not 2-colorable**
Desargues	20	30	2	2	40	11	7	1	proper
C_{30} (cycle)	30	30	2	3	90	11	8	0	proper
C_{21} (odd)	21	21	3	2	42	12	17	9	**not 2-colorable**
`myciel3`	11	20	4	4	44	10	4	0	proper
				3	33	11	158	83	**not 3-colorable**
`myciel4`	23	71	5	5	115	11	11	0	proper

Table 5
Certified decomposition. First group: k above χ to scale nk (colorable direction only); second: the same graphs at $k = \chi$ and $k = \chi - 1$; third: further graphs; fourth: the DIMACS Mycielski graphs. "Max piece" is the largest number of slots in a simulated piece, in which a precolored vertex has one slot and a free vertex k (so $K_{1,12}$ at $k = 10$ gives $1 + 10 = 11$). "Calls" counts device solves (each ≤ 13 logical slots); "exact" counts the leaves on which the device returned no coloring and which the exact check then decided. All of these leaves were infeasible, so no colorable leaf was missed. Colorings are device witnesses checked classically; every "not colorable" verdict is a classical certificate (Theorem 8.5 (ii)) accepted by the verified checker of Section 9.8. Raw data: `benchmarks/results/largenk.csv`, `decomp.csv` and `dimacs.csv`.

runs are certified by their colorings; the 7 non-colorable ones by refutations of 2 to 794 nodes: 2–3 nodes for the 1-colorability checks, 49 for Petersen and 79 for the dodecahedral graph at $k = 2$, 95 for C_{21} at $k = 2$, and 794 for `myciel3` at $k = 3$, whose certificate file is under 7 KB. Together with the 4-coloring found on the device, this refutation makes $\chi(\texttt{myciel3}) = 4$ a machine-checkable fact whose lower bound does not depend on any quantum or simulated component. The design is a certifying algorithm McConnell et al. (2011): the device searches for colorings, classical certificates establish non-colorability, and in both directions trust rests on a checker proved correct in Rocq rather than on the solver.

10. Discussion and limitations

What is and is not guaranteed. The reduction is exact, and its output can be verified: a readout that passes the check of Section 4.3 is a certified coloring, whatever the quality of the solve. Two things are not guaranteed: that the device reaches the ground state, and that a large instance fits. Theorem 4.6 separates both from correctness, and on the combinatorial gadget a solve that misses the ground state still decodes within the bound of Theorem 4.10. A device failure is never read as a lower bound: every non-colorability verdict in this paper either matches the oracle or is certified by a refutation that the verified checker accepts. Section 9.1 states the scope of the mechanization.

The size wall. For devices of a few hundred atoms Wurtz et al. (2023), the $O((nk)^2)$ atom count is the binding constraint; under the Exponential Time Hypothesis, a quadratic overhead is unavoidable for reductions of this kind Cazals et al. (2025). With the reference construction, only very small instances meet

a 256-atom budget ($nk \leq 8$ in Table 4, and $nk = 12$ for a path), and the classical simulator stops near 16 slots. Three techniques reduce the size: preprocessing; line ordering, which makes the atom count of bounded-pathwidth families linear in n; and separator decomposition, which keeps every simulated solve at ≤ 13 logical slots while nk reaches 192. Applied before gadgetization, decomposition is the route to device-sized instances, since pieces with $wk \leq 8$ meet a 256-atom budget. It helps, however, only when balanced separators are small; dense instances such as the queen graphs and `myciel5` remain out of reach, and our largest certified run is on `myciel4`.

Simulation, not hardware. Every dynamics result in this paper is a classical simulation: at most 16 slots of G'_k with the blockade imposed on the edges of G'_k, instances $A_4(G'_k)$ of at most 12 vertices, and one unweighted five-atom unit-disk instance on the `bloqade-analog` emulator. The metric gadget instances are laid out and checked exactly, but not simulated; for them the offset identity rests on Nguyen et al. (2023) and on the exact checks of Section 9.5. We have not mapped the gadget weights γ, 2γ, 4γ and $\gamma \pm w_F(v)/2$ onto a device: on Aquila, they would have to be expressed through the static pattern $h_v \in [0, 1]$ that scales one shared local detuning (Section 2). This mapping, and any run on hardware, remain open.

Scope: the analog regime. For a gate-based target Graham et al. (2022), Stage 1 would carry over unchanged, but geometric realization would become qubit routing, and Theorem 5.5 would have no analogue. A sampled output whose quality depends on circuit depth would replace the single sweep, which would give the verifiability argument of Remark 4.7 more weight. The type system does not depend on this choice, because it types reductions rather than Hamiltonians.

11. Future work

Robustness for the metric gadget. Theorem 4.10 covers the combinatorial gadget. Its metric counterpart needs a lower bound on the weight that a defect in a copy line or crossing gadget of Nguyen et al. (2023) costs; with such a bound, a solve ε below the optimum would contain $O(\varepsilon)$ defects and decode, after local repair, to a coloring with $k+O(\varepsilon)$ colors. Whether the constants of the existing gadgets support this is open.

Translation validation of the emitted instances. The certificate checkers validate each verdict. The analogous step for compilation is to validate each emitted metric instance: check that its lattice coordinates realize exactly the prescribed adjacency, prove the local MWIS identity of each gadget by exhaustive computation, and compose the gadgets by a gluing lemma. Each compiled instance, rather than the compiler, would then carry the proof. Three further items belong to this line of work: linking the explicit model of G'_k in `Robust`, which already carries the choice cliques as edges, to the pipeline's interface sort; mechanizing the restrictions of Proposition 4.3; and proving type preservation and progress for the surface elaboration.

Decomposition before gadgetization. Proposition 4.12 makes the atom count of bounded-pathwidth inputs linear in n, and the separator decomposition keeps every piece small. Gadgetizing each piece separately, together with its certificate, is therefore a direct route to device-sized instances. The choice of gadget set, either the construction of Nguyen et al. (2023) unchanged or a custom one, determines what must be proved again.

Hardware. The first step is to map the gadget weights onto the local-detuning controls of a device; the second is to measure, on QuEra-class hardware Wurtz et al. (2023), the ground-state success probabilities and the gap distribution that Theorem 4.10 converts into extra colors. The size wall limits which instances such experiments can use.

12. Conclusion

In its analog regime with the standard two-level encoding, neutral-atom hardware solves UDG–MWIS, not coloring, so an exact coloring compiler must reduce coloring to UDG–MWIS. We do so by composing the classical colorability→independent-set reduction with copy/crossing gadgetization. The resulting pipeline runs in polynomial time, does not inflate the chromatic number, has an exact decoder, and is proved correct once for all inputs. The geometric heuristics on which a supergraph approach would rely provably cannot bound chromatic inflation, so we use them only in the layout optimizer, where they cannot affect correctness. On this separation we built a certified-reduction type system whose derivations give soundness and the legal

stage order. We extended it with graded decoders, under which a solve of the combinatorial gadget that is ε below the optimum costs at most ε extra colors. We also paired the device with a verified checker, so that colorings come from the device and lower bounds from classical refutations. The calculus, the combinatorial gadget, the checker and the end-to-end robustness theorem are mechanized axiom-free in Rocq. The binding limitation is size. The gadget instance is quadratic in general and linear only for bounded pathwidth, so present hardware reaches only very small instances; decomposition before gadgetization is the route beyond them.

A. Exactly realizable families for the stand-alone supergraph backend

This appendix concerns only the supergraph backend of Section 5.3, which we keep for comparison. It asks which graph families embed as unit-disk graphs with *zero* false edges. One might expect every caterpillar (a tree whose non-leaf vertices induce a path) to be a 1D unit-disk graph, but this is false. Stars are caterpillars, yet $K_{1,3}$ is not a unit-interval graph, and $K_{1,6}$ is not a unit-disk graph even in the plane (Proposition 5.4). The expectation confuses interval graphs with unit-interval graphs, and one dimension with two.

Proposition A.1 (Trees as 1D unit-disk and interval graphs). *A tree is a 1D unit-disk graph (equivalently, a unit-interval or proper-interval graph) if and only if it is a path. A tree is an interval graph if and only if it is a caterpillar.*

Proof. Points on a line with threshold r correspond to intervals of length r centered at them, and equal-length intervals cannot contain one another, so the 1D unit-disk graphs are exactly the unit-interval, equivalently proper-interval, graphs Roberts (1969); Golumbic (1980), which have no induced $K_{1,3}$ Roberts (1969). In a tree, a vertex of degree at least 3 and three of its neighbors induce a $K_{1,3}$, so such a tree has maximum degree at most 2 and is a path; conversely, a path is realized at $0, 1, 2, \ldots$ with $r = 1$. The characterization of interval trees as caterpillars is classical Lekkerkerker and Boland (1962); Golumbic (1980).

Together with Proposition 5.4, which realizes exactly the stars $K_{1,t}$ with $t \leq 5$, these are the families on which the backend is exact.

Data availability

The complete artifact is publicly available at `https://github.com/slant14/Graph-colouring`: the Rocq development with the extracted certificate checker (`proof/`), the Haskell package `udg-color` and the Rydberg simulator (`src/`), and the benchmark scripts with all recorded results and certificates (`benchmarks/`). The metric layouts use Julia 1.11 with UnitDiskMapping.jl 0.5.1 QuEra Computing Inc. (2025), and the other benchmarks are Python scripts; together they reproduce every numeric result of Section 9.

Acknowledgments

The authors thank Nikolai Kudasov for ideas, feedback, and a careful review of this paper, all of which improved the work. Any remaining errors are our own.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used Claude (Anthropic) to assist with language revision, manuscript organization, and improvement of textual clarity. The tool was not used to generate research results, analyses, or conclusions. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the published article.

References

Angkhanawin, T., Deger, A., Pritchard, J.D., Adams, C.S., 2026. Graph coloring via quantum optimization on a Rydberg-qudit atom array. Quantum Science and Technology 11, 025012. doi:10.1088/2058-9565/ae3b6d.

Arnborg, S., Proskurowski, A., 1989. Linear time algorithms for NP-hard problems restricted to partial k-trees. Discrete Applied Mathematics 23, 11–24. doi:10.1016/0166-218X(89)90031-0.

Belkin, M., Niyogi, P., 2001. Laplacian eigenmaps and spectral techniques for embedding and clustering, in: Advances in Neural Information Processing Systems 14 (NIPS 2001), MIT Press. pp. 585–591. doi:10.7551/mitpress/1120.003.0080.

Blass, A., 1995. Questions and answers – a category arising in linear logic, complexity theory, and set theory, in: Girard, J.Y., Lafont, Y., Regnier, L. (Eds.), Advances in Linear Logic. Cambridge University Press. volume 222 of *London Mathematical Society Lecture Note Series*, pp. 61–82. doi:10.1017/CBO9780511629150.004.

Bodlaender, H.L., 1996. A linear-time algorithm for finding tree-decompositions of small treewidth. SIAM Journal on Computing 25, 1305–1317. doi:10.1137/S0097539793251219.

Bomze, I.M., Budinich, M., Pardalos, P.M., Pelillo, M., 1999. The maximum clique problem, in: Du, D.Z., Pardalos, P.M. (Eds.), Handbook of Combinatorial Optimization. Springer, Boston, MA, pp. 1–74. doi:10.1007/978-1-4757-3023-4_1.

Brattka, V., Gherardi, G., 2011. Weihrauch degrees, omniscience principles and weak computability. The Journal of Symbolic Logic 76, 143–176. doi:10.2178/jsl/1294170993.

Brélaz, D., 1979. New methods to color the vertices of a graph. Communications of the ACM 22, 251–256. doi:10.1145/359094.359101.

Breu, H., Kirkpatrick, D.G., 1998. Unit disk graph recognition is NP-hard. Computational Geometry: Theory and Applications 9, 3–24. doi:10.1016/S0925-7721(97)00014-X.

Cazals, P., Sorondo, A., Onofre, V., Dalyac, C., Coelho, W., Vitale, V., 2025. Quantum optimization on Rydberg-atom arrays with arbitrary connectivity: Gadget limitations and a heuristic approach. Physical Review A 112, 062416. doi:10.1103/3sjz-gfmr.

Cerezo, M., Arrasmith, A., Babbush, R., Benjamin, S.C., Endo, S., Fujii, K., McClean, J.R., Mitarai, K., Yuan, X., Cincio, L., Coles, P.J., 2021. Variational quantum algorithms. Nature Reviews Physics 3, 625–644. doi:10.1038/s42254-021-00348-9.

Chung, F.R.K., 1997. Spectral Graph Theory. volume 92 of *CBMS Regional Conference Series in Mathematics*. American Mathematical Society, Providence, RI. doi:10.1090/cbms/092.

Clark, B.N., Colbourn, C.J., Johnson, D.S., 1990. Unit disk graphs. Discrete Mathematics 86, 165–177. doi:10.1016/0012-365X(90)90358-O.

Crescenzi, P., 1997. A short guide to approximation preserving reductions, in: Proceedings of the Twelfth Annual IEEE Conference on Computational Complexity, IEEE. pp. 262–273. doi:10.1109/CCC.1997.612321.

Dirac, G.A., 1961. On rigid circuit graphs. Abhandlungen aus dem Mathematischen Seminar der Universität Hamburg 25, 71–76. doi:10.1007/BF02992776.

Ebadi, S., Keesling, A., Cain, M., Wang, T.T., Levine, H., Bluvstein, D., Semeghini, G., Omran, A., Liu, J.G., Samajdar, R., Luo, X.Z., Nash, B., Gao, X., Barak, B., Farhi, E., Sachdev, S., Gemelke, N., Zhou, L., Choi, S., Pichler, H., Wang, S.T., Greiner, M., Vuletić, V., Lukin, M.D., 2022. Quantum optimization of maximum independent set using Rydberg atom arrays. Science 376, 1209–1215. doi:10.1126/science.abo6587.

Ebadi, S., Wang, T.T., Levine, H., Keesling, A., Semeghini, G., Omran, A., Bluvstein, D., Samajdar, R., Pichler, H., Ho, W.W., Choi, S., Sachdev, S., Greiner, M., Vuletić, V., Lukin, M.D., 2021. Quantum phases of matter on a 256-atom programmable quantum simulator. Nature 595, 227–232. doi:10.1038/s41586-021-03582-4.

Fejes Tóth, L., 1953. Lagerungen in der Ebene, auf der Kugel und im Raum. Springer, Berlin. doi:10.1007/978-3-662-01206-2.

Fiedler, M., 1973. Algebraic connectivity of graphs. Czechoslovak Mathematical Journal 23, 298–305. doi:10.21136/CMJ.1973.101168.

Forster, Y., Larchey-Wendling, D., Dudenhefner, A., Heiter, E., Kirst, D., Kunze, F., Smolka, G., Spies, S., Wehr, D., Wuttke, M., 2020. A Coq library of undecidable problems, in: The Sixth International Workshop on Coq for Programming Languages (CoqPL 2020), New Orleans, LA, USA. URL: https://popl20.sigplan.org/details/CoqPL-2020-papers/5/A-Coq-Library-of-Undecidable-Problems.

Fruchterman, T.M.J., Reingold, E.M., 1991. Graph drawing by force-directed placement. Software: Practice and Experience 21, 1129–1164. doi:10.1002/spe.4380211102.

Gaboardi, M., Katsumata, S.y., Orchard, D., Breuvart, F., Uustalu, T., 2016. Combining effects and coeffects via grading, in: Proc. ICFP 2016, pp. 476–489. doi:10.1145/2951913.2951939.

Gäher, L., Kunze, F., 2021. Mechanising complexity theory: The Cook-Levin theorem in Coq, in: Cohen, L., Kaliszyk, C. (Eds.), 12th International Conference on Interactive Theorem Proving (ITP 2021), Schloss Dagstuhl – Leibniz-Zentrum für Informatik. pp. 20:1–20:18. doi:10.4230/LIPIcs.ITP.2021.20.

Garey, M.R., Johnson, D.S., 1979. Computers and Intractability: A Guide to the Theory of NP-Completeness. W. H. Freeman, San Francisco.

Glover, F., Laguna, M., 1997. Tabu Search. Kluwer Academic Publishers, Boston. doi:10.1007/978-1-4615-6089-0.

Golumbic, M.C., 1980. Algorithmic Graph Theory and Perfect Graphs. Academic Press, New York.

Graham, T.M., Song, Y., Scott, J., Poole, C., Phuttitarn, L., Jooya, K., Eichler, P., Jiang, X., Marra, A., Grinkemeyer, B., Kwon, M., Ebert, M., Cherek, J., Lichtman, M.T., Gillette, M., Gilbert, J., Bowman, D., Ballance, T., Campbell, C., Dahl, E.D., Crawford, O., Blunt, N.S., Rogers, B., Noel, T., Saffman, M., 2022. Multi-qubit entanglement and algorithms on a neutral-atom quantum computer. Nature 604, 457–462. doi:10.1038/s41586-022-04603-6.

Hammack, R., Imrich, W., Klavžar, S., 2011. Handbook of Product Graphs. 2nd ed., CRC Press, Boca Raton. doi:10.1201/b10959.

Henriet, L., Beguin, L., Signoles, A., Lahaye, T., Browaeys, A., Reymond, G.O., Jurczak, C., 2020. Quantum computing with neutral atoms. Quantum 4, 327. doi:10.22331/q-2020-09-21-327.

Hietala, K., Rand, R., Hung, S.H., Wu, X., Hicks, M., 2021. A verified optimizer for quantum circuits. Proceedings of the ACM on Programming Languages 5, 37:1–37:29. doi:10.1145/3434318.

Igarashi, A., Kobayashi, N., 2002. Resource usage analysis, in: Proceedings of the 29th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), ACM. pp. 331–342. doi:10.1145/503272.503303.

Johnson, D.S., Trick, M.A. (Eds.), 1996. Cliques, Coloring, and Satisfiability: Second DIMACS Implementation Challenge. volume 26 of *DIMACS Series in Discrete Mathematics and Theoretical Computer Science*. AMS. doi:10.1090/dimacs/026.

Jung, H.W.E., 1901. Über die kleinste Kugel, die eine räumliche Figur einschliesst. Journal für die reine und angewandte Mathematik 123, 241–257. doi:10.1515/crll.1901.123.241.

Kang, R.J., Müller, T., 2012. Sphere and dot product representations of graphs. Discrete & Computational Geometry 47, 548–568. doi:10.1007/s00454-012-9394-8.

Karp, R.M., 1972. Reducibility among combinatorial problems, in: Miller, R.E., Thatcher, J.W. (Eds.), Complexity of Computer Computations. Plenum Press, New York, pp. 85–103. doi:10.1007/978-1-4684-2001-2_9.

Katsumata, S.y., 2014. Parametric effect monads and semantics of effect systems, in: Proc. POPL 2014, pp. 633–645. doi:10.1145/2535838.2535846.

Kim, M., Kim, K., Hwang, J., Moon, E.G., Ahn, J., 2022. Rydberg quantum wires for maximum independent set problems. Nature Physics 18, 755–759. doi:10.1038/s41567-022-01629-5.

Kinnersley, N.G., 1992. The vertex separation number of a graph equals its path-width. Information Processing Letters 42, 345–350. doi:10.1016/0020-0190(92)90234-M.

Kirkpatrick, S., Gelatt, C.D., Vecchi, M.P., 1983. Optimization by simulated annealing. Science 220, 671–680. doi:10.1126/science.220.4598.671.

Komar, P., Bylinskii, A., Becker, D., Lin, M., 2024. Local detuning now available on QuEra's Aquila device with Braket Direct. AWS Quantum Technologies Blog, 17 April 2024, https://aws.amazon.com/blogs/quantum-computing/local-detuning-now-available-on-queras-aquila-device-with-braket-direct.

Kruskal, J.B., 1964. Multidimensional scaling by optimizing goodness of fit to a nonmetric hypothesis. Psychometrika 29, 1–27. doi:10.1007/BF02289565.

Lekkerkerker, C.G., Boland, J.C., 1962. Representation of a finite graph by a set of intervals on the real line. Fundamenta Mathematicae 51, 45–64. doi:10.4064/fm-51-1-45-64.

Leroy, X., 2009. Formal verification of a realistic compiler. Communications of the ACM 52, 107–115. doi:10.1145/1538788.1538814.

Levin, L.A., 1973. Universal sequential search problems. Problems of Information Transmission 9, 265–266. Russian original: Problemy Peredachi Informatsii 9(3):115–116, 1973.

Lovász, L., 1978. Kneser's conjecture, chromatic number, and homotopy. Journal of Combinatorial Theory, Series A 25, 319–324. doi:10.1016/0097-3165(78)90022-5.

Lucassen, J.M., Gifford, D.K., 1988. Polymorphic effect systems, in: Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL), ACM. pp. 47–57. doi:10.1145/73560.73564.

Madsen, L.S., Laudenbach, F., Askarani, M.F., Rortais, F., Vincent, T., Bulmer, J.F.F., Miatto, F.M., Neuhaus, L., Helt, L.G., Collins, M.J., Lita, A.E., Gerrits, T., Nam, S.W., Vaidya, V.D., Menotti, M., Dhand, I., Vernon, Z., Quesada, N., Lavoie, J., 2022. Quantum computational advantage with a programmable photonic processor. Nature 606, 75–81. doi:10.1038/s41586-022-04725-x.

Marathe, M.V., Breu, H., Hunt, III, H.B., Ravi, S.S., Rosenkrantz, D.J., 1995. Simple heuristics for unit disk graphs. Networks 25, 59–68. doi:10.1002/net.3230250205.

McConnell, R.M., Mehlhorn, K., Näher, S., Schweitzer, P., 2011. Certifying algorithms. Computer Science Review 5, 119–161. doi:10.1016/j.cosrev.2010.09.009.

Miettinen, K., 1999. Nonlinear Multiobjective Optimization. International Series in Operations Research & Management Science, Kluwer Academic Publishers, Boston. doi:10.1007/978-1-4615-5563-6.

Moser, L., Moser, W., 1961. Solution to problem 10. Canadian Mathematical Bulletin 4, 187–189. doi:10.1017/S0008439500025765.

Mycielski, J., 1955. Sur le coloriage des graphs. Colloquium Mathematicum 3, 161–162. doi:10.4064/cm-3-2-161-162.

Nguyen, M.T., Liu, J.G., Wurtz, J., Lukin, M.D., Wang, S.T., Pichler, H., 2023. Quantum optimization with arbitrary connectivity using Rydberg atom arrays. PRX Quantum 4, 010316. doi:10.1103/PRXQuantum.4.010316.

Nielson, F., Nielson, H.R., 1999. Type and effect systems, in: Olderog, E.R., Steffen, B. (Eds.), Correct System Design: Recent Insights and Advances. Springer, Berlin, Heidelberg. volume 1710 of *Lecture Notes in Computer Science*, pp. 114–136. doi:10.1007/3-540-48092-7_6.

de Oliveira, A.G., Diamond-Hitchcock, E., Walker, D.M., Wells-Pestell, M.T., Pelegrí, G., Picken, C.J., Malcolm, G.P.A., Daley, A.J., Bass, J., Pritchard, J.D., 2025. Demonstration of weighted-graph optimization on a Rydberg-atom array using local light shifts. PRX Quantum 6, 010301. doi:10.1103/PRXQuantum.6.010301.

Orchard, D., Liepelt, V.B., Eades III, H., 2019. Quantitative program reasoning with graded modal types. Proceedings of the ACM on Programming Languages 3, 110:1–110:30. doi:10.1145/3341714.

Papadimitriou, C.H., Yannakakis, M., 1991. Optimization, approximation, and complexity classes. Journal of Computer and System Sciences 43, 425–440. doi:10.1016/0022-0000(91)90023-X.

Pichler, H., Wang, S.T., Zhou, L., Choi, S., Lukin, M.D., 2018. Quantum optimization for maximum independent set using Rydberg atom arrays. arXiv preprint arXiv:1808.10816 doi:10.48550/arXiv.1808.10816.

Pnueli, A., Siegel, M., Singerman, E., 1998. Translation validation, in: Proc. TACAS 1998, Springer. pp. 151–166. doi:10.1007/BFb0054170.

Poljak, S., 1974. A note on stable sets and colorings of graphs. Commentationes Mathematicae Universitatis Carolinae 15, 307–309. URL: https://dml.cz/handle/10338.dmlcz/105554.

Preskill, J., 2018. Quantum computing in the NISQ era and beyond. Quantum 2, 79. doi:10.22331/q-2018-08-06-79.

QuEra Computing Inc., 2025. UnitDiskMapping.jl: reductions from combinatorial problems to unit-disk maximum independent set problems. https://github.com/QuEraComputing/UnitDiskMapping.jl. Version 0.5.1, reference implementation of the construction of Nguyen et al. (2023).

Roberts, F.S., 1969. Indifference graphs, in: Harary, F. (Ed.), Proof Techniques in Graph Theory. Academic Press, New York, pp. 139–146.

Robertson, N., Seymour, P.D., 1986. Graph minors. II. Algorithmic aspects of tree-width. Journal of Algorithms 7, 309–322. doi:10.1016/0196-6774(86)90023-4.

Saffman, M., Walker, T.G., Mølmer, K., 2010. Quantum information with Rydberg atoms. Reviews of Modern Physics 82, 2313–2363. doi:10.1103/RevModPhys.82.2313.

Scholl, P., Schuler, M., Williams, H.J., Eberharter, A.A., Barredo, D., Schymik, K.N., Lienhard, V., Henry, L.P., Lang, T.C., Lahaye, T., Läuchli, A.M., Browaeys, A., 2021. Quantum simulation of 2D antiferromagnets with hundreds of Rydberg atoms. Nature 595, 233–238. doi:10.1038/s41586-021-03585-1.

Simon, J., 1975. On some central problems in computational complexity. Ph.D. thesis. Cornell University. Ithaca, NY. Technical Report TR75-224, Department of Computer Science.

Talpin, J.P., Jouvelot, P., 1992. Polymorphic type, region and effect inference. Journal of Functional Programming 2, 245–271. doi:10.1017/S0956796800000393.

Tan, Y.K., Heule, M.J.H., Myreen, M.O., 2021. cake_lpr: Verified propagation redundancy checking in CakeML, in: Proc. TACAS 2021, Springer. pp. 223–241. doi:10.1007/978-3-030-72013-1_12.

Tarjan, R.E., 1985. Decomposition by clique separators. Discrete Mathematics 55, 221–232. doi:10.1016/0012-365X(85)90051-2.

Tutte, W.T., 1954. A contribution to the theory of chromatic polynomials. Canadian Journal of Mathematics 6, 80–91. doi:10.4153/CJM-1954-010-9.

Valiant, L.G., 1979. The complexity of enumeration and reliability problems. SIAM Journal on Computing 8, 410–421. doi:10.1137/0208032.

Volpano, D., Irvine, C., Smith, G., 1996. A sound type system for secure flow analysis. Journal of Computer Security 4, 167–187. doi:10.3233/JCS-1996-42-304.

Welsh, D.J.A., Powell, M.B., 1967. An upper bound for the chromatic number of a graph and its application to timetabling problems. The Computer Journal 10, 85–86. doi:10.1093/comjnl/10.1.85.

West, D.B., 2001. Introduction to Graph Theory. 2nd ed., Prentice Hall, Upper Saddle River, NJ.

Wetzler, N., Heule, M.J.H., Hunt, Jr., W.A., 2014. DRAT-trim: Efficient checking and trimming using expressive clausal proofs, in: Proc. SAT 2014, Springer. pp. 422–429. doi:10.1007/978-3-319-09284-3_31.

Wurtz, J., Bylinskii, A., Braverman, B., Amato-Grill, J., Cantu, S.H., Huber, F., Lukin, A., Liu, F., Weinberg, P., Long, J., Wang, S.T., Gemelke, N., Keesling, A., 2023. Aquila: QuEra's 256-qubit neutral-atom quantum computer. arXiv preprint arXiv:2306.11727 doi:10.48550/arXiv.2306.11727.

Zuckerman, D., 2007. Linear degree extractors and the inapproximability of max clique and chromatic number. Theory of Computing 3, 103–128. doi:10.4086/toc.2007.v003a006.